

武汉大学计算机学院

软件工程

课程实验（设计）报告

组 号： B260

项目名称： WikiBridge 零代码本地知识库 API 分享与 Open-Code/MCP 消费平台

专业（班）： 软件工程（特软班）

项目组成员：组长： SES27-张沐雷-2024302121024

成员： SES27-张沐雷-2024302121024：组织工作、BearFRP 前端、跨端联调、持续集成 CI、桌面端测试/自动化相关

SES02-陈明德-2024300001010：OpenCode + MCP 接入、跨端联调、BearFRP 相关开发

SES42-骆贝尔-2024302131247：LLM-Wiki 方向

SES03-郭一言-2024300002015：Wiki 方向、frp 模块及联调相关工作

SES38-曾浩正-2024302131092：OpenCode / Chat-box 方向

SES23-童济舟-2024302111261：OpenCode / Chat-box 方向

任课教师： 杜卓敏

2026 年 6 月 26 日

目 录

第一部分	计划任务书	
第二部分	需求规格说明书	
第三部分	软件设计说明书	
第四部分	测试说明书	
第五部分	个人总结报告	
第六部分	小组个人贡献评分表	

第一部分

计划任务书

(终版)

文档变更历史记录

序号	变更日期	变更人员	变更内容详情描述	变更后的版本号
1	2026 年 6 月 1 日	项目组	启动文档编写，形成初始项目背景、范围和阶段性内容。	V0.1
2	2026 年 6 月 18 日	项目组	补充核心功能、架构、测试任务和阶段性实现材料。	V0.9
3	2026 年 6 月 26 日	项目组	根据最终联调结果整理终版文档并纳入分组实习报告，补充测试方法、缺陷矩阵和个人贡献评分表。	V1.0

文档编号：WikiBridge-PLAN-1.0

WikiBridge

计划任务书

零代码本地知识库 API 分享与 OpenCode/MCP 消费平台

项目名称：WikiBridge 零代码本地知识库 API 分享与 OpenCode/MCP 消费平台

项目代号：WikiBridge

文档版本：V1.0

参考模板：计划任务书，参考 GB8567-88 与 GB856T-88 相关要求

编写人员：WikiBridge 项目组

适用阶段：课程设计立项、计划与验收评审

日期：2026 年 6 月 26 日

文档变更历史记录

序号	日期	版本	变更人员	变更内容
1	2026 年 6 月 1 日	V0.1	项目组	启动项目选题与可行性分析, 形成问题背景、项目目标、初始范围和本地知识库分享场景。
2	2026 年 6 月 10 日	V0.2	项目组	补充 B 端、S 端、C 端角色、核心功能、投入收益、风险和项目开发计划草案。
3	2026 年 6 月 18 日	V0.3	项目组	补充计划书级用例图、架构图、WBS、甘特图和交付物清单, 形成计划任务书主体。
4	2026 年 6 月 25 日	V0.4	项目组	根据 MCP 架构调整收敛方案: 淘汰 “B 端发布完整 OpenCode”, 明确 B 端只发布 LLM-Wiki API, C 端本地 OpenCode + MCP 消费远程知识库。
5	2026 年 6 月 26 日	V1.0	项目组	收敛计划任务书图表边界, 保留用例、架构演进、WBS 和甘特图, 并按软件设计说明书版式整理为 LaTeX 正式版。

文档约定

- 本文档为《计划任务书》，内容覆盖可行性分析、项目开发计划和课程项目组织安排，结构参考 GB/T 8567-88 与 GB/T 856T-88，并结合课程项目实际进行裁剪。
- 文档面向课程指导教师、项目组成员、验收评委和后续维护者。
- 文档中的人员分工、项目周期和里程碑日期以 2026 年 6 月 26 日终版材料为准。
- 本文档采用当前联调后确定的最终架构：B 端发布 LLM-Wiki API, S 端提供 BearFRP/frps 控制面和公网入口，C 端本地运行 OpenCode 并通过 MCP 访问远程知识库。

目录

文档变更历史记录 I

文档约定 I

1 引言 1

1.1 编写目的 1

1.2 项目背景 1

1.3 项目相关方 1

1.4 与其他系统的关系 2

1.5 术语与缩写 2

1.6 参考资料 3

2 项目概述 3

2.1 产品定位 3

2.2 范围与边界 4

2.2.1 In-Scope 4

2.2.2 Out-of-Scope 4

2.3 核心功能概览 4

2.4 项目可交付物 5

2.4.1 程序 5

2.4.2 文档 5

2.4.3 服务与演示材料 5

3 需求分析 6

3.1 用户角色与画像 6

3.2 用户用例图 6

3.3 典型用例 7

3.3.1 UC-01 B 端构建本地知识库 7

3.3.2 UC-02 B 端发布 LLM-Wiki API 7

3.3.3 UC-03 C 端添加远程知识库 7

3.3.4 UC-04 C 端本地 OpenCode + MCP 对话 8

3.4 功能性需求 8

3.5	需求追踪关系	9
3.6	非功能性需求	9
3.6.1	安全与隐私	9
3.6.2	可靠性	10
3.6.3	可维护性	10
3.6.4	可用性	10
4	系统设计	10
4.1	架构演进	10
4.2	当前总体技术路线	11
4.3	关键设计决策	12
4.4	安全边界设计	12
4.5	部署计划	12
5	可行性分析	13
5.1	技术可行性	13
5.2	经济可行性	14
5.3	法律与合规可行性	14
5.4	社会与运行可行性	14
5.5	操作可行性	15
6	备选方案比较	15
6.1	备选方案 1: B 端发布完整 OpenCode	15
6.2	备选方案 2: B 端只发布静态 Wiki 页面	15
6.3	备选方案 3: B 端发布 LLM-Wiki API, C 端本地 OpenCode + MCP	16
6.4	方案对比总结	16
7	投入与效益分析	17
7.1	项目投入	17
7.2	项目收益	17
7.3	风险与敏感性	17
8	项目开发计划	18
8.1	工作内容综述	18

8.2	工作分解结构	18
8.3	人员组织与分工	19
8.4	协作机制	20
8.5	进度安排	20
8.6	验收标准	21
8.6.1	功能完整度	21
8.6.2	安全边界	21
8.6.3	测试要求	21
8.6.4	文档要求	22
8.7	风险与应对	22
9	支持条件	22
9.1	开发环境	22
9.2	测试环境	23
9.3	运行环境	23
9.4	组内支持条件	23
10	结论与建议	23
A	终版材料索引	24
B	可引用证据	24

1、引言

1.1 编写目的

本文档的目的有三点：

1. 从技术、经济、法律合规、运行和操作等角度论证 WikiBridge 项目的可行性，为课程项目立项和后续开发提供依据。
2. 明确项目目标、范围边界、核心需求和关键架构决策，作为后续需求规格说明书、软件设计规格说明书和测试文档的输入。
3. 给出项目工作分解、人员组织、进度安排、交付物和风险应对计划，便于项目组执行、跟踪和复盘。

1.2 项目背景

个人和小团队在学习、科研、课程设计、项目协作中会积累大量本地资料，例如 PDF、Markdown、网页剪藏、课堂笔记、研究记录和项目文档。这些资料通常分散在本地文件夹和不同工具中，难以沉淀为可浏览、可检索、可问答的知识库。

现有知识库方案通常存在两类门槛：

- 部署门槛高：用户需要理解后端服务、向量数据库、模型 API、网页托管和公网访问配置。
- 分享边界不清：将资料上传到云端平台虽然方便，但对一些课程资料、科研资料或未公开项目资料并不合适；本地保存虽然安全，但很难临时分享给同学、老师或小组成员访问。

WikiBridge 的核心目标是让知识拥有者在本地生成和持有知识库，并通过受控公网入口分享给特定访客。项目不是公共博客平台，也不是云端全量搜索引擎，而是面向“本地知识库生成 + 有限共享 + Agent 化消费”的课程项目原型。

1.3 项目相关方

相关方	角色描述
项目提出者	WikiBridge 项目组。
开发者	WikiBridge 项目组，共 6 人，按 LLM-Wiki、BearFRP/frp、OpenCode/Chatbox、Desktop、MCP、CI 与测试等方向协作。

B 端用户	知识库拥有者和发布者，负责本地资料整理、Wiki 构建和分享入口创建。
S 端服务	BearFRP 控制面和 frps 公网转发层，负责账号、鉴权、子域名/端口和隧道状态。
C 端用户	知识消费者或受限访客，通过本地 OpenCode + MCP 访问 B 端分享的知识库 API。
外部依赖	frp/frpc/frps、OpenCode、LLM-Wiki、模型 API、Docker Compose、GitHub Actions。

1.4 与其他系统的关系

WikiBridge 不是从零实现所有能力，而是集成和改造多个现有组件：

- LLM-Wiki：负责本地资料编译、Wiki 页面生成、项目和文件 API。
- OpenCode：作为 C 端本地知识消费和对话式访问入口。
- MCP server：将 B 端分享的 LLM-Wiki API 注册为 C 端 OpenCode 可调用的工具。
- BearFRP/frp：负责把 B 端本地 LLM-Wiki API 通过公网入口发布给 C 端。
- WikiBridge Desktop：负责统一桌面端操作、远程知识库添加、本地 OpenCode 启动和 MCP 注册。

1.5 术语与缩写

术语	含义
B 端	知识库拥有者/发布者所在环境，保存本地资料和 Wiki 内容。
S 端	公网服务器端，运行 BearFRP 控制面和 frps。
C 端	知识消费者/受限访客所在环境，本地运行 OpenCode。
LLM-Wiki	本地知识库编译与 API 服务，提供项目、文件、搜索等能力。
OpenCode	C 端本地知识消费与对话界面。
MCP	Model Context Protocol，用于将 LLM-Wiki 能力接入 Open-Code。
BearFRP	本项目中的 frp 控制面，负责账号、代理、流量和发布入口。

frpc/frps	frp 客户端/服务端，用于内网穿透和公网访问转发。
KB 模式	OpenCode 的知识库模式，限制文件访问和工具能力范围。
API 分享	B 端通过 BearFRP 分享 LLM-Wiki API，而不是分享完整 OpenCode 页面。

1.6 参考资料

1. GB/T 8567-88 《计算机软件开发文件编制指南》。
2. GB/T 856T-88 《项目开发计划编写规范》。
3. frp 官方项目：<https://github.com/fatedier/frp>。
4. OpenCode 项目资料与本项目集成代码。
5. LLM-Wiki 项目资料与本项目集成代码。
6. WikiBridge 部署说明。
7. WikiBridge Desktop 交接说明。
8. 旧方案架构对比图。
9. 新方案架构对比图。

2、项目概述

2.1 产品定位

WikiBridge 定位为“本地优先的知识库生成与有限共享工具”。其核心命题是：让知识拥有者在本地生成和持有 Wiki，通过 BearFRP 发布受控 API，再由 C 端本地 OpenCode + MCP 进行对话式知识消费。

项目差异化定位包括：

- 本地优先：原始资料、Wiki 页面和主要知识内容保留在 B 端本地。
- 有限共享：B 端主动分享访问入口，C 端作为受限访客访问指定知识库。
- 受控 Agent 化消费：OpenCode 不运行在 B 端公网暴露环境，而是在 C 端本地运行，通过 MCP 访问 B 端分享的 LLM-Wiki API。
- 低部署门槛：用户不需要手写 frp 配置和复杂网络脚本，系统负责生成和连接发布入口。

2.2 范围与边界

2.2.1 In-Scope

- 本地资料管理和 Wiki 构建流程。
- LLM-Wiki 项目、文件、搜索和健康检查 API。
- BearFRP 用户注册、登录、充值、代理创建、脚本生成、HTTP/TCP 发布入口。
- B 端通过 BearFRP 发布 LLM-Wiki API。
- C 端桌面端添加远程知识库 API 地址。
- C 端本地启动 OpenCode，并注册 llm-wiki-* MCP。
- OpenCode KB 模式下只读访问、MCP 白名单和基本安全边界控制。
- Docker Compose 一键集成部署和基本演示链路。
- 基础测试、系统 mock、CI 回归和课程验收文档。

2.2.2 Out-of-Scope

- 面向全网的公开博客、CDN 分发和搜索引擎收录。
- 云端托管用户原始资料、Wiki 正文或问答上下文。
- 大规模多租户商业运营、计费 and SLA。
- 移动端原生 App。
- 完整企业级权限体系、审计平台和内容审核流程。
- 真实支付、商业化订阅和长期运维。
- 对任意 OpenCode 工具能力的公网开放。

2.3 核心功能概览

功能模块	关键能力
本地知识库构建	导入资料、生成 Markdown Wiki、维护 Wiki 项目和文件。
LLM-Wiki API	健康检查、项目列表、文件读取、搜索、图谱/元数据接口。
BearFRP 控制面	用户、余额、代理、子域名/端口、frpc 脚本、在线状态。
发布链路	B 端 LLM-Wiki API 经 frpc/frps 发布为公网 URL。
C 端远程知识库	添加远程 API 地址、检测可用性、选择项目、保存 token。

OpenCode MCP	+ C 端本地 OpenCode 启动、MCP 注册、对话中调用远程知识库。
安全边界	KB 只读模式、llm-wiki-* MCP 白名单、禁止暴露 B 端完整 Agent。
部署与验证	Docker Compose、健康检查、CI 测试、演示文档。

2.4 项目可交付物

2.4.1 程序

- WikiBridge Desktop 桌面端源码。
- LLM-Wiki 服务源码及知识库 API。
- BearFRP 控制面、frps/frpc 集成代码。
- OpenCode KB 模式与 MCP 接入相关修改。
- LLM-Wiki MCP server。
- Docker Compose 部署配置。
- GitHub Actions / CI 配置。
- 系统测试 mock 和测试用例。

2.4.2 文档

- 《WikiBridge 计划任务书》（本文档）。
- 《WikiBridge 需求规格说明书》。
- 《WikiBridge 软件设计规格说明书》。
- 《WikiBridge 测试文档/测试报告》。
- 《WikiBridge 部署说明》。
- 《WikiBridge Desktop 交接说明》。
- 答辩 PPT、演示截图和项目总结材料。

2.4.3 服务与演示材料

- 本地 Docker Compose 演示栈。
- BearFRP 公网发布入口。
- C 端本地 OpenCode + MCP 远程知识库演示。
- 架构对比图：旧方案与新方案架构对比材料。

3、需求分析

3.1 用户角色与画像

角色	画像描述
B 端知识拥有者	有本地课程资料、科研资料或项目资料，希望生成结构化 Wiki，并临时分享给同学、老师、导师或小组成员。
C 端知识消费者	被授权访问某个知识库的访客，希望浏览、搜索和对话式提问，但不应拥有 B 端本机权限。
S 端管理员	维护 BearFRP 控制面、frps、部署环境、端口/子域名和安全凭据。
项目开发者	负责 LLM-Wiki、BearFRP、OpenCode/MCP、桌面端、测试与文档。

3.2 用户用例图

下图给出 WikiBridge 计划阶段的顶层用例。它只表达各类参与者与核心业务目标之间的关系，不展开界面按钮和内部 API 细节。

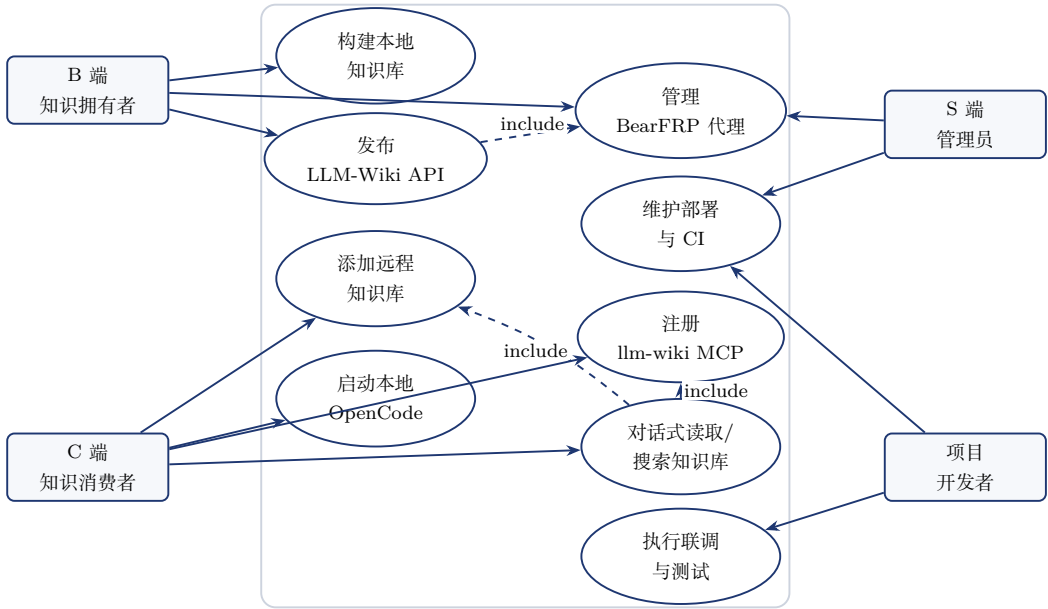


图 1: WikiBridge 用户用例图

3.3 典型用例

3.3.1 UC-01 B 端构建本地知识库

项目	内容
主参与者	B 端知识拥有者。
前置条件	用户已准备本地资料，LLM-Wiki 服务可用。
基本路径	添加资料 → 触发构建 → LLM-Wiki 生成 Markdown Wiki → 本地可浏览。
扩展路径	模型配置缺失时提示配置；构建失败时保留错误日志。
后置条件	本地生成 Wiki 项目和页面。

3.3.2 UC-02 B 端发布 LLM-Wiki API

项目	内容
主参与者	B 端知识拥有者。
前置条件	BearFRP 可用，B 端 LLM-Wiki API 可访问。
基本路径	登录 BearFRP → 创建 HTTP/TCP 代理 → 启动 frpc → 获得公网 API URL。
扩展路径	通配域名未配置时提示使用 TCP 或配置真实域名；余额不足时拒绝创建。
后置条件	C 端可通过公网 URL 访问 B 端 LLM-Wiki API。

3.3.3 UC-03 C 端添加远程知识库

项目	内容
主参与者	C 端知识消费者。
前置条件	C 端已安装或启动 WikiBridge Desktop，持有 B 端分享的 API URL。
基本路径	输入远程 API URL → 检查 <code>/api/v1/health</code> 和项目列表 → 保存远程知识库。
扩展路径	URL 不完整、服务不可达、token 错误时给出明确提示。
后置条件	远程知识库进入 C 端列表，可选择项目。

3.3.4 UC-04 C 端本地 OpenCode + MCP 对话

项目	内容
主参与者	C 端知识消费者。
前置条件	C 端保存模型 API Key，远程知识库可访问。
基本路径	启动本地 OpenCode → 注册 llm-wiki-* MCP → 创建 KB Chat Session → 对话中读取/搜索远程知识库。
扩展路径	OpenCode 未就绪、MCP 注册失败、远程 API 401/403 时提示处理方式。
后置条件	C 端可通过本地 OpenCode 对 B 端知识库进行对话式消费。

3.4 功能性需求

编号	需求描述	优先级
FR-01	系统应支持 B 端将本地资料构建为 Markdown Wiki 项目。	P0
FR-02	系统应提供 LLM-Wiki API，支持健康检查、项目列表、文件读取和搜索。	P0
FR-03	系统应支持 BearFRP 创建 TCP/HTTP 发布代理，并生成可运行 frpc 配置或脚本。	P0
FR-04	系统应支持 B 端发布 LLM-Wiki API 公网入口。	P0
FR-05	系统应在 HTTP 子域名模式下依赖真实可解析通配域名，配置不满足时给出提示或降级建议。	P0
FR-06	C 端应能添加远程知识库 API URL，并检测服务可达性和项目列表。	P0
FR-07	C 端应能本地启动 OpenCode，并创建 KB Chat Session。	P0
FR-08	C 端应能注册 llm-wiki-* MCP，使 OpenCode 能调用远程 LLM-Wiki API。	P0
FR-09	OpenCode 在 KB 模式下应限制危险能力，只允许受控知识库访问。	P0
FR-10	系统应支持远程知识库 token 配置，以访问受保护 API。	P1
FR-11	系统应提供 Docker Compose 集成部署和健康检查。	P1
FR-12	系统应提供基本自动化测试和 CI 验证远程知识库流程。	P1

FR-13 系统应提供交接说明、部署说明和验收演示材料。

P1

3.5 需求追踪关系

需求编号	来源用例	验证方式
FR-01	UC-01	LLM-Wiki 构建流程测试、手工导入资料验证。
FR-02	UC-01、UC-03、UC-04	/api/v1/health、项目列表、文件读取和搜索接口测试。
FR-03	UC-02	BearFRP 代理创建、脚本生成和在线状态测试。
FR-04	UC-02	B/S/C 分布式联调，公网 URL 访问健康检查。
FR-05	UC-02	HTTP 子域名配置测试，未配置通配域名时验证错误提示。
FR-06	UC-03	Desktop 远程知识库添加 flow 系统测试。
FR-07	UC-04	OpenCode 启动和 KB Session 创建测试。
FR-08	UC-04	MCP 注册状态和工具调用测试。
FR-09	UC-04	KB 只读模式、MCP 白名单和危险能力阻断测试。
FR-10	UC-03、UC-04	token 配置、401/403 错误处理测试。
FR-11	UC-02、UC-04	Docker Compose 集成启动和健康检查。
FR-12	UC-03、UC-04	CI 中 desktop system tests 和 mock 回归。
FR-13	全部用例	文档审查、演示材料和交接检查。

3.6 非功能性需求

3.6.1 安全与隐私

- S 端不得保存用户原始资料、Wiki 正文或问答上下文。
- B 端不应通过 frp 暴露完整 OpenCode Agent 环境。
- C 端模型供应商、API Key 和模型名应保存在 C 端本地。
- OpenCode KB 模式应默认阻断危险文件访问、终端执行和不受控工具能力。
- MCP 接入应通过白名单限制，仅允许 WikiBridge LLM-Wiki MCP。

3.6.2 可靠性

- B 端 API 发布后，C 端应能通过 `/api/v1/health` 进行健康检查。
- BearFRP 应能正确维护代理状态、子域名/端口和在线状态。
- 远程知识库不可达时，桌面端应给出明确错误提示，而不是静默失败。
- CI 应覆盖桌面端远程知识库 flow 的核心 mock 和系统测试。

3.6.3 可维护性

- 后端接口应使用结构化模型和清晰错误响应。
- 关键配置通过环境变量或桌面端配置管理，避免硬编码。
- 重要跨模块接口应有交接文档和验证命令。
- 大型功能提交应尽量拆分为多个小提交，便于 Code Review 和回滚。

3.6.4 可用性

- B 端用户不应手写复杂 frpc 配置。
- C 端用户只需要粘贴远程知识库 API 地址并保存自己的模型 API Key。
- 远程知识库连接、MCP 状态和 OpenCode 启动状态应在界面上可见。

4、系统设计

4.1 架构演进

项目经历了一次关键架构调整。旧方案设想是：B 端同时运行 LLM-Wiki 和 OpenCode，然后通过 frp 将完整 OpenCode 页面发布给 C 端访问。该方案在联调中暴露出三个问题：

1. 模型供应商、API Key、本地配置归属混乱。
2. OpenCode 具备文件访问和工具调用能力，直接通过 frp 暴露会扩大 B 端安全风险。
3. C 端访客的对话行为运行在 B 端环境，B/C 端责任边界不清。

最终调整为：B 端只发布 LLM-Wiki API；C 端本地运行 OpenCode，并通过 MCP 调用远程知识库 API。新方案使三端职责更清晰：

- B 端负责知识库生成、持有和 API 分享。
- S 端负责 BearFRP 控制面、frps 转发、子域名/端口和流量调度。

- C 端负责模型配置、本地 OpenCode Agent 和对话式知识消费。

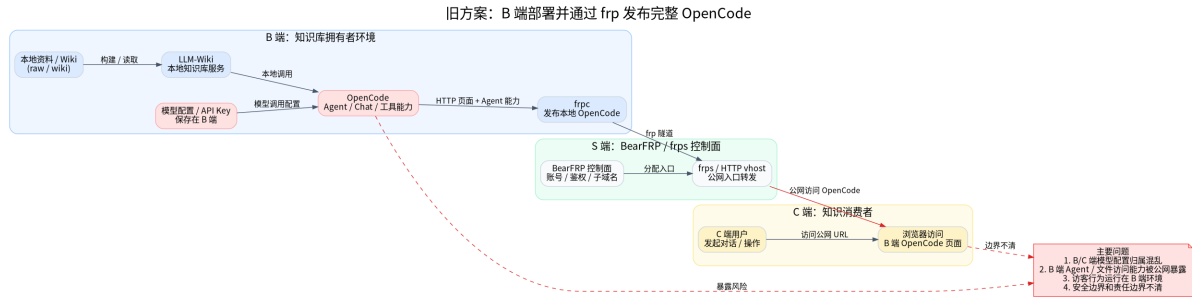


图 2：旧方案架构图：B 端发布完整 OpenCode

新方案：B 端发布 LLM-Wiki API，C 端本地 OpenCode + MCP 消费知识

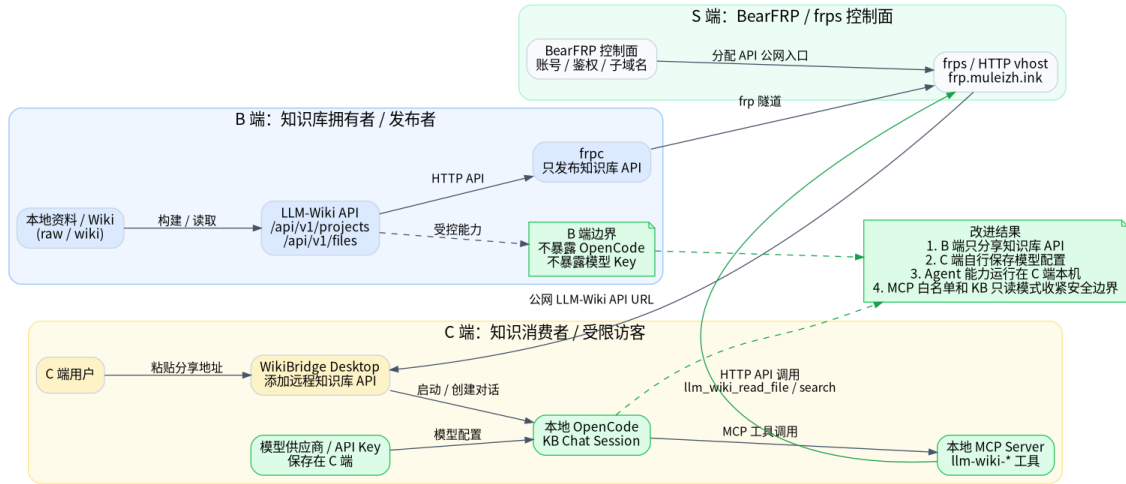


图 3：新方案架构图：C 端本地 OpenCode + MCP

4.2 当前总体技术路线

计划阶段只记录高层技术路线，详细数据流、实体关系、时序图、活动图和部署拓扑迁入后续《软件设计规格说明书》。当前主链路如下：

1. B 端本地资料经 LLM-Wiki 构建为 Wiki 项目。
2. B 端通过 BearFRP/frpc 发布 LLM-Wiki API。
3. S 端 BearFRP/frps 提供公网入口，例如 *.frp.muleizh.in。
4. C 端在 WikiBridge Desktop 中添加远程知识库 API URL。
5. C 端本地启动 OpenCode。
6. C 端注册 llm-wiki-* MCP。
7. OpenCode 对话调用 llm_wiki_read_file、llm_wiki_search 等工具访问远程知识库。

4.3 关键设计决策

决策点	选型	主要理由
知识内容存放	B 端本地	保持本地优先，S 端不托管知识正文。
公网访问	BearFRP/frp	适合将本地 API 临时映射到公网。
C 端知识消费	本地 OpenCode + MCP	保持模型配置和 Agent 环境在 C 端。
OpenCode 安全	KB 只读 + MCP 白名单	避免不受控工具能力暴露。
部署方式	Docker Compose + Desktop sidecar	课程项目周期内可控，方便演示和联调。
HTTP 发布模式	优先真实通配域名；无域名时 TCP 模式	避免生成不可解析的子域名 URL。

4.4 安全边界设计

OpenCode + MCP 接入的关键安全措施包括：

- B 端不发布完整 OpenCode，只发布 LLM-Wiki API。
- C 端保存自己的模型 API Key，不依赖 B 端模型配置。
- KB 模式下 OpenCode 只允许知识库范围内的受控访问。
- OPENCODE_KB_READONLY 下拒绝写入、修改和删除。
- llm-wiki-* MCP 白名单限制 MCP 名称、命令、入口文件和环境变量。

相关实现证据包括 OpenCode KB guard、OpenCode MCP 注册模块、桌面端 sidecar 模块和桌面端联调说明。

4.5 部署计划

项目支持两类运行方式：

1. Docker Compose 集成演示栈：用于本地和课程验收演示。
2. B/S/C 分布式联调：B 端运行 LLM-Wiki API，S 端运行 BearFRP/frps，C 端运行 WikiBridge Desktop 和 OpenCode。

关键端口和服务如下：

服务	默认端口	说明
LLM-Wiki API	19828	B 端知识库 API。
OpenCode	4096	C 端本地 OpenCode。
BearFRP backend	8000	S 端控制面。
frps bind	7000	frpc 连接入口。
HTTP vhost	8080	HTTP 子域名发布入口。
Desktop dev server	1420	Tauri 前端开发入口。

5、可行性分析

5.1 技术可行性

技术点	成熟度评估	团队储备/风险
LLM-Wiki	中高	已具备本地服务和 API，需继续完善构建流程和测试。
OpenCode	高	上游成熟，但本项目需要 KB 模式和 MCP 接入改造。
MCP	中高	协议适合工具接入，但需要处理注册、白名单和错误提示。
BearFRP/frp	高	frp 成熟，BearFRP 控制面需要联调和边界测试。
Tauri/React Desktop	中高	能完成桌面端整合，但跨平台 CI 仍需修复。
Docker Compose	高	适合课程演示和多服务编排。
GitHub Actions CI	高	已有桌面端系统测试,但当前远程知识库 flow 仍有失败项。

关键技术风险与缓解：

- HTTP 子域名发布依赖真实通配域名：通过 `frp.muleizh.ink` 配置或 TCP 模式规避。

- OpenCode 暴露风险：通过架构调整避免 B 端发布 OpenCode。
- MCP 工具能力过宽：通过 llm-wiki-* 白名单和 KB 只读模式限制。
- 跨平台桌面测试不稳定：后续修复 CI 和 system mocks。

5.2 经济可行性

作为课程项目，现金投入较低，主要成本是开发人力和少量服务器/域名资源。

项目	估算	说明
云服务器/VPS	0-300 元	可使用学生额度或组内已有服务器。
域名	0-100 元/年	HTTP 子域名模式建议配置真实域名。
模型 API	0-少量	开发期可使用测试额度。
CI/GitHub	0	使用免费额度。
人力成本	不计现金	课程项目投入。

若产品化，收益不宜按广告或订阅直接估算。更合理的价值在于降低本地知识库分享门槛，保护未公开资料和科研/课程资料的本地持有权，并支持小组协作、导师临时查看、课程项目交付等有限共享场景。

5.3 法律与合规可行性

项目应遵守最小化数据收集和本地优先原则：

- S 端只保存账号、鉴权、发布记录、隧道状态等控制面数据。
- 不在 S 端存储用户原始资料、Wiki 正文或问答上下文。
- C 端模型 API Key 不上传到 S 端。
- 对外发布入口应支持关闭、回收和 token 保护。
- 公开部署前必须修改默认密码、frps token 和管理员凭据。

5.4 社会与运行可行性

项目适合以下场景：

- 课程小组临时共享项目资料和知识库。
- 科研小组分享未公开资料的结构化浏览入口。
- 个人将本地笔记整理成 Wiki 后分享给同学或导师查看。

- 答辩和演示场景中展示本地知识库的公网访问能力。

项目不适合面向全网的公开内容传播、高并发长期内容托管，以及要求搜索引擎收录、评论互动、CDN 分发的博客平台。

5.5 操作可行性

B 端用户需要理解的操作应控制在添加资料、构建 Wiki、创建/查看发布入口、分享 API URL。C 端用户需要理解的操作应控制在输入模型 API Key、粘贴远程知识库 API URL、点击连接并在 OpenCode 中提问。因此，系统必须避免让普通用户直接理解 frpc/frps、HTTP vhost、MCP 配置文件等内部细节。

6、备选方案比较

6.1 备选方案 1：B 端发布完整 OpenCode

该方案最初被考虑：B 端运行完整 OpenCode，通过 frp 将 OpenCode 页面发布给 C 端访问。

优势：

- 实现路径直观。
- C 端无需安装或启动 OpenCode。
- 页面访问链路较短。

劣势：

- B/C 端模型配置归属混乱。
- B 端 Agent、文件访问和工具能力被公网暴露。
- C 端访客行为运行在 B 端环境。
- 安全边界不清。

结论：不采用。

6.2 备选方案 2：B 端只发布静态 Wiki 页面

该方案只发布渲染后的 Wiki 页面，不提供 C 端 OpenCode/MCP 对话能力。

优势：

- 安全风险最低。
- 实现简单。

- 对 C 端要求低，只需浏览器。

劣势：

- 缺少对话式知识消费能力。
- 项目亮点不足。
- 不能体现 OpenCode/MCP 集成价值。

结论：可作为降级方案，但不是主方案。

6.3 备选方案 3：B 端发布 LLM-Wiki API，C 端本地 OpenCode + MCP

该方案为当前主方案。

优势：

- B 端不暴露完整 OpenCode。
- C 端保存自己的模型配置。
- Agent 能力运行在 C 端本地。
- B/S/C 三端职责清晰。
- 可通过 MCP 白名单和 KB 只读模式收紧安全边界。

劣势：

- C 端需要启动本地 OpenCode。
- MCP 注册、桌面端状态管理和错误提示更复杂。
- 自动化测试和 CI 覆盖成本更高。

结论：采用。

6.4 方案对比总结

维度	B 端发布 Open-Code	只发布静态 Wiki	C 端 OpenCode + MCP
安全边界	差	优	良好
C 端体验	中	中	优
Agent 能力	有但风险高	无	有且边界清晰
实现复杂度	中	低	高

课程技术深度	中	低	高
最终选择	不采用	降级方案	主方案

7、投入与效益分析

7.1 项目投入

投入项	估算	说明
服务器	0-300 元	BearFRP/frps 需要公网服务器。
域名	0-100 元	HTTP 子域名模式推荐配置通配域名。
模型 API	少量	开发和演示阶段可控。
开发人力	6 人课程小组	按 LLM-Wiki、BearFRP/frp、Open-Code/Chatbox、Desktop、MCP、CI 与测试等方向投入。
测试与文档	课程实验周期内完成	覆盖实验 5、实验 6、终版分组报告和个人总结材料。

7.2 项目收益

- 完成本地知识库生成、发布和 C 端对话式消费闭环。
- 验证 BearFRP + LLM-Wiki + OpenCode + MCP 的集成可行性。
- 形成可展示的课程项目原型。
- 沉淀 B/S/C 三端架构、MCP 接入和 KB 安全边界设计经验。
- 为个人和小团队的有限知识共享提供原型参考。

7.3 风险与敏感性

风险	影响	缓解措施	说明
通配域名未配置	HTTP 子域名 URL 不可解析	配置真实域名，或使用 TCP 发布模式。	影响演示入口可用性。

OpenCode 直接暴露	B 端安全风险扩大	采用 C 端本地 OpenCode + MCP。	已通过架构调整规避。
模型 API 配置缺失	C 端无法对话	桌面端提供模型配置入口和明确提示。	需在验收前验证。
MCP 注册失败	OpenCode 无法访问远 程知识库	检查 MCP 状态、提供重试和错误提示。	属于核心链路风险。
CI 跨平台失败	质量保障不完整	修复 desktop system mocks，补回归测试。	需保留手工验证路径。
frpc/frps 连接不稳定	公网入口不可用	健康检查、日志、重连和手工验证。	与网络环境相关。
默认凭据未修改	公开部署安全风险	公开部署前强制修改密码和 token。	发布前必须处理。

8、项目开发计划

8.1 工作内容综述

项目开发工作按六个阶段组织：

- 1. 需求与架构确认。
- 2. 原型与核心模块开发。
- 3. BearFRP 发布链路开发。
- 4. LLM-Wiki / OpenCode / MCP 集成。
- 5. 端到端联调与测试。
- 6. 文档、演示和验收交付。

8.2 工作分解结构

阶段	工作内容	关键产物
需求与架构	用户角色、B/S/C 架构、边界和 风险确认。	SRS、架构图、计划书。

知识库模块	LLM-Wiki 构建、API、项目/文 件/搜索能力。
发布控制面	BearFRP 用户、代理、余额、脚 本、frps/frpc。
桌面端集成	WikiBridge Desktop、远程知识 库、OpenCode 启停。
OpenCode + MCP	KB 模式、MCP 注册、只读和白 名单。
部署与 CI	Docker Compose、sidecar、系统 测试、CI。
测试与文档	黑盒/白盒、联调缺陷、验收材 料。

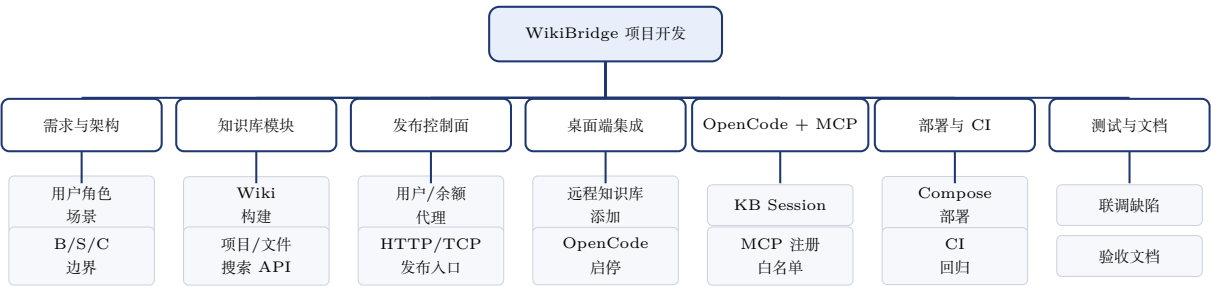


图 4: WikiBridge 工作分解结构图

8.3 人员组织与分工

以下分工按终版联调和提交材料归纳，只记录主要方向，实际开发中存在跨模块讨论、联调和共同修正。

成员	主要方向	说明
陈明德	OpenCode + MCP 接入、跨 端联调、BearFRP 相关开发	已完成 C 端本地 OpenCode + MCP 访问远程 LLM-Wiki API 的 联调。
郭一言	Wiki 方向、frp 模块及联调 相关工作	负责 Wiki 相关集成, 并参与 frp 发 布链路联调。
张沐雷	持续集成 CI、桌面端测试/自 动化相关	负责 GitHub Actions 持续集成、桌 面端测试和自动化测试支撑。

骆贝尔	LLM-Wiki 方向	负责 LLM-Wiki 相关功能和知识库生成能力。
曾浩正	OpenCode / Chatbox 方向	负责 OpenCode / Chatbox 相关知识消费能力。
童济舟	OpenCode / Chatbox 方向	参与 OpenCode / Chatbox 方向开发与联调。

8.4 协作机制

- 代码仓库：Git/GitHub。
- 沟通方式：线下讨论、IM 群组和 Git 提交记录。
- 分支策略：当前仍需规范化，建议后续采用 feature branch + 小粒度提交。
- Code Review：目前没有正式 PR Review，后续建议至少对大功能合并做轻量评审。
- 测试策略：联调测试 + 系统测试 + CI 回归 + 实验 5 黑盒/白盒测试。

8.5 进度安排

以下按课程项目实际推进节奏整理。

阶段	周期	里程碑
M1 需求与计划	W1	完成计划书、SRS 初稿、分工确认。
M2 架构设计	W2	完成 SDS、B/S/C 架构图、关键接口设计。
M3 核心开发	W3–W5	完成 LLM-Wiki、BearFRP、Desktop、OpenCode/MCP 核心开发。
M4 联调演示	W5–W6	跑通 B 端 API 发布、C 端 OpenCode + MCP 访问链路。
M5 测试完成	W6	完成实验 5 黑盒/白盒测试和 CI 修复。
M6 验收交付	W7	完成测试文档、部署说明、答辩 PPT、个人总结。

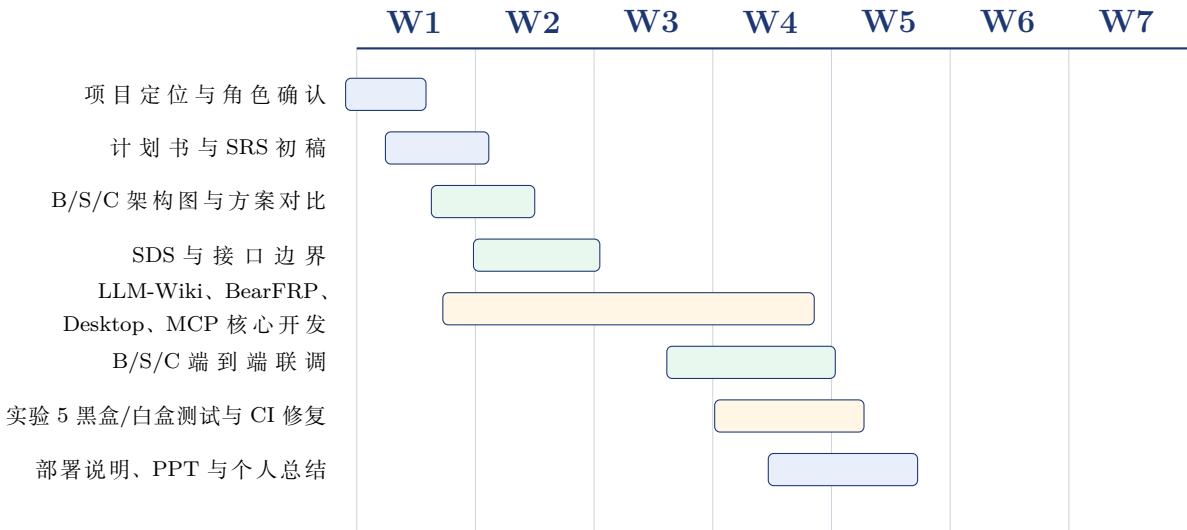


图 5: WikiBridge 项目周级甘特图

8.6 验收标准

8.6.1 功能完整度

- B 端能生成本地 Wiki。
- B 端能通过 BearFRP 发布 LLM-Wiki API。
- C 端能添加远程知识库 API。
- C 端能启动本地 OpenCode 并注册 llm-wiki-* MCP。
- C 端能在 OpenCode 对话中读取或搜索远程知识库。

8.6.2 安全边界

- B 端不暴露完整 OpenCode。
- C 端模型 API Key 不上传到 S 端。
- OpenCode KB 模式和 MCP 白名单生效。
- 公开部署前默认密码和 token 已替换。

8.6.3 测试要求

- BearFRP 用户与代理测试覆盖注册、登录、充值、代理创建、删除和脚本生成。
- 桌面端系统测试覆盖远程知识库 flow。
- 实验 5 补充黑盒测试和白盒控制流图。
- CI 中关键测试通过或对失败项有明确修复计划。

8.6.4 文档要求

- 计划书、需求规格说明书、软件设计规格说明书、测试文档、部署说明和交接说明齐备。
- 架构图体现旧方案与新方案对比。
- 个人总结能引用截图、提交号、测试编号和文件路径。

8.7 风险与应对

风险描述	可能性	影响	应对措施	责任人
HTTP 子域名缺少通配域名	中	高	配置真实域名 <code>frp.muleizh.ink</code> 或切换 TCP 发布。	发布链路负责人
B 端 OpenCode 暴露风险	中	高	改为 C 端本地 OpenCode + MCP。	OpenCode 与 MCP 负责人
MCP 注册失败	中	中	增加状态展示、错误提示和重试。	Desktop 与 OpenCode 负责人
CI 跨平台失败	中	中	修复 mocks, 拆分测试, 保留手工验证路径。	CI 负责人
提交粒度过粗	中	中	后续按功能拆分提交, 轻量 Review。	全体
文档滞后	高	中	计划书、SRS、SDS 与测试文档并行补齐。	文档负责人
模型 API 不可用	中	中	提供配置入口和错误提示, 保留无模型降级说明。	LLM-Wiki 与 OpenCode 负责人

9、支持条件

9.1 开发环境

- 操作系统: Linux / macOS / Windows。
- IDE: VS Code 或 JetBrains 系列。

- Node.js: 项目当前桌面端开发使用 Node 22 相关环境。
- Rust/Tauri: 用于桌面端。
- Python/FastAPI: 用于 BearFRP 控制面。
- Docker Compose: 用于集成部署。
- Git/GitHub: 用于版本管理和 CI。

9.2 测试环境

- 本地 Docker Compose 环境。
- GitHub Actions 桌面端 CI。
- BearFRP 公网服务器和真实域名。
- 本地 C 端 OpenCode 环境。

9.3 运行环境

- B 端: 运行 LLM-Wiki API、frpc、知识库数据目录。
- S 端: 运行 BearFRP backend、frps、控制面数据。
- C 端: 运行 WikiBridge Desktop、OpenCode、MCP server。

9.4 组内支持条件

- 需要组内统一最终分工表。
- 需要补齐 SRS 和 SDS。
- 需要为实验 5 预留黑盒/白盒测试时间。
- 需要在验收前确认所有截图、Git 记录、CI 状态和部署说明。

10、结论与建议

综合分析，WikiBridge 在课程项目周期内具备可行性。项目所依赖的 LLM-Wiki、OpenCode、MCP、frp、Docker Compose 等技术均有可用基础，核心难点不在单点技术是否存在，而在跨模块集成、三端职责边界和安全控制。

项目已经明确从旧方案“B 端发布完整 OpenCode”调整为新方案“B 端发布 LLM-Wiki API，C 端本地 OpenCode + MCP”。该调整降低了 B 端安全风险，明确了模型配置归属，也更符合有限共享的产品定位。

建议后续重点推进：

1. 补齐需求规格说明书和软件设计规格说明书，将 B/S/C 三端最终架构写入正式文档。
2. 完成实验 5 黑盒/白盒测试，特别是远程知识库、OpenCode 启动、MCP 注册和 BearFRP 发布链路。
3. 修复桌面端远程知识库 flow 的 CI 失败项。
4. 规范提交粒度和轻量 Code Review，避免大功能一次性提交影响追溯。
5. 在课程项目中期或类似项目后续迭代中增加最小端到端链路演示，允许展示半成品但要求说明已闭环、mock 和待完善内容。

A、终版材料索引

- WikiBridge 软件需求规格说明书。
- WikiBridge 软件设计说明书。
- WikiBridge 测试分析报告。
- WikiBridge 终版分组实习报告。

B、可引用证据

证据	说明
WikiBridge Desktop 交接说明	当前架构说明。
WikiBridge 部署说明	部署和 BearFRP 发布说明。
端到端联调截图	C 端本地 OpenCode 调用 llm_wiki_read_file 读取远程知识库。
远程知识库流程 CI 截图	远程 Wiki flow 系统测试 CI 失败截图。
bab897616	fix:enableOpenCodeKBchatsessions。
62e049539	fix:sharellmwikiapifordesktopremotes。
816554778	docs:adddesktophandoffstartupguide。

cee791768

test:updatedesktopsystemmocksforremotewikiflow。

第二部分

需求规格说明书

(终版)

文档变更历史记录

序号	变更日期	变更人员	变更内容详情描述	变更后的版本号
1	2026 年 6 月 1 日	项目组	启动文档编写，形成初始项目背景、范围和阶段性内容。	V0.1
2	2026 年 6 月 18 日	项目组	补充核心功能、架构、测试任务和阶段性实现材料。	V0.9
3	2026 年 6 月 26 日	项目组	根据最终联调结果整理终版文档并纳入分组实习报告，补充测试方法、缺陷矩阵和个人贡献评分表。	V1.0

文档编号: WikiBridge-SRS-1.0

WikiBridge

软件需求规格说明书

零代码本地知识库 API 分享与 OpenCode/MCP 消费平台

项目名称: WikiBridge 零代码本地知识库 API 分享与 OpenCode/MCP 消费平台

项目代号: WikiBridge

文档版本: V1.0

架构基线: SDS V1.2: B 端发布 LLM-Wiki API, S 端提供 BearFRP/frps, C 端本地 OpenCode + MCP

编写人员: WikiBridge 项目组

适用阶段: 课程设计原型、需求评审、系统验收与后续维护

日期: 2026 年 6 月 26 日

文档变更历史记录

序号	变更日期	变更人员	版本	变更内容详情描述
1	2026 年 6 月 1 日	项目组	V0.1	启动需求调研，形成项目背景、用户角色、本地知识库分享场景和初始范围说明。
2	2026 年 6 月 10 日	项目组	V0.2	完善 B 端、S 端、C 端职责划分，整理核心功能范围、边界约束和非功能需求初稿。
3	2026 年 6 月 18 日	项目组	V0.3	完善顶层用例、功能需求编号、异常流程和验收口径，形成需求规格说明书主体结构。
4	2026 年 6 月 25 日	项目组	V0.9	根据联调结论调整 MCP 架构：淘汰“B 端发布完整 OpenCode”方案，确立“B 端发布 LLM-Wiki API，C 端本地 OpenCode + MCP 消费远程知识库”需求基线。
5	2026 年 6 月 26 日	项目组	V1.0	对照计划书、设计说明书和测试材料完善功能需求、非功能需求、验收标准和原型演示证据，形成正式版。

目录

文档变更历史记录 I

1 引言 1

1.1 编写目的 1

1.2 读者对象 1

1.3 软件项目概述 1

1.4 文档概述 2

1.5 定义 2

1.6 参考资料 3

2 软件的一般性描述 3

2.1 软件产品与其环境之间的关系 3

2.2 三端角色与职责 4

2.3 假设与前提条件 5

2.4 产品范围边界 5

2.4.1 范围内 5

2.4.2 范围外 6

2.5 架构需求演进 6

3 软件功能需求描述 6

3.1 软件需求的用例模型 6

3.2 关键业务流程 7

3.3 核心用例表 7

3.4 功能需求概览 8

3.5 软件需求的分析模型与功能需求详述 9

3.5.1 FR-01 本地资料导入与 Wiki 构建 9

3.5.2 FR-02 LLM-Wiki API 健康检查、项目、文件、搜索能力 10

3.5.3 FR-03 B 端通过 BearFRP 发布 LLM-Wiki API 11

3.5.4 FR-04 S 端代理、隧道、端口/子域名和状态管理 11

3.5.5 FR-05 C 端添加远程知识库 API 12

3.5.6 FR-06 C 端本地启动 OpenCode 13

- 3.5.7 FR-07 注册并调用 llm-wiki MCP 工具 13
 - 3.5.8 FR-08 对话式读取、搜索和总结远程知识库 14
 - 3.5.9 FR-09 KB 模式安全边界 15
 - 3.5.10 FR-10 部署、健康检查、联调和测试支撑 16
- 3.6 需求追踪矩阵 17
- 4 其它软件需求描述 17**
 - 4.1 性能要求 17
 - 4.2 安全与隐私要求 18
 - 4.3 可用性与可靠性要求 19
 - 4.4 兼容性与运行环境要求 19
 - 4.5 界面要求 19
 - 4.6 进度要求 20
 - 4.7 交付要求 20
 - 4.8 验收要求 21
- 5 软件原型与演示证据 21**
 - 5.1 原型界面文字描述 21
 - 5.2 已完成截图证据 21
 - 5.3 演示脚本 22

1、引言

1.1 编写目的

本文档定义 WikiBridge 系统在课程设计阶段应满足的功能性需求、非功能性需求、运行边界和验收口径。它的目的包括：

1. 在项目组、课程指导教师、评审人员、测试人员和后续维护者之间建立统一需求基线。
2. 明确 B 端、S 端、C 端三类环境的职责划分，避免把实现方案误解为用户需求。
3. 为软件设计说明书、测试文档、演示脚本和最终验收提供可追踪依据。
4. 记录架构需求演进：旧方案“B 端发布完整 OpenCode”因模型配置归属不清和 Agent 能力暴露风险被淘汰；当前需求基线要求 B 端只发布 LLM-Wiki API，C 端本地运行 OpenCode 并通过 MCP 消费远程知识库。

本文档描述“系统应当做什么”和“验收时如何判断完成”，不展开类设计、详细部署拓扑、内部时序和数据库实现细节。上述设计内容以《WikiBridge 软件设计规格说明书》为准。

1.2 读者对象

本文档主要面向：

- 武汉大学软件工程课程指导教师、答辩评审人员和项目验收方。
- WikiBridge 项目组成员，包括需求、设计、开发、测试、部署和文档负责人。
- 后续维护者，用于理解系统边界、核心功能和安全约束。
- 演示与测试人员，用于准备端到端联调、截图证据和验收材料。

1.3 软件项目概述

WikiBridge 是一个本地优先的知识库生成、API 分享和受控 Agent 化消费平台。项目面向学生、教师、研究者和小团队，解决本地资料难以结构化沉淀、AI 知识库部署门槛高、本地知识库难以有限分享、Agent 能力公网暴露边界不清等问题。

系统将知识库生成、发布和消费拆分为三个环境：

- B 端：知识库所有者/发布者所在环境，保存本地资料、Wiki 内容和 LLM-Wiki API。
- S 端：公网服务器端，运行 BearFRP 控制面和 frps，负责账号、代理、隧道和公网入口。

- C 端：知识消费者所在环境，本地运行 WikiBridge Desktop、OpenCode 和 llm-wiki MCP server，保存自己的模型供应商、模型名和 API Key。
- 当前需求基线的核心业务是：B 端在本地生成 Wiki 后，通过 BearFRP/frp 发布受控 LLM-Wiki API；C 端在本地添加该远程知识库 API，启动本地 OpenCode 并注册 llm_wiki_* MCP 工具，以对话方式读取、搜索和总结远程知识库。

1.4 文档概述

本文档共五章。第一章为引言，说明编写目的、读者对象、项目概述、术语定义和参考资料。第二章为软件的一般性描述，说明产品运行环境、三端角色、假设前提、范围边界和架构演进约束。第三章为软件功能需求描述，给出用例模型、关键业务流程、功能需求编号和核心用例表。第四章为其它软件需求描述，说明性能、安全、可用性、兼容性、界面、进度、交付和验收要求。第五章为软件原型与演示证据，描述当前原型界面和已完成截图证据位置。

1.5 定义

术语 / 缩写	释义
WikiBridge	本项目名称，表示连接本地知识库、受控公网入口和本地 Agent 消费端的桥接平台。
B 端	知识库所有者/发布者所在环境，保存本地资料、Wiki 页面和 LLM-Wiki API。
S 端	公网服务器端，运行 BearFRP 控制面、frps 和 HTTP vhost/TCP 发布入口。
C 端	知识消费者所在环境，本地运行 WikiBridge Desktop、OpenCode 和 MCP server。
LLM-Wiki	本地知识库编译与 API 服务，提供健康检查、项目列表、文件读取、搜索、图谱等能力。
OpenCode	C 端本地对话式知识消费入口，负责模型调用、KB Chat Session 和工具调用。
MCP	Model Context Protocol，用于把远程 LLM-Wiki API 注册为 OpenCode 可调用工具。
BearFRP	本项目中的 frp 控制面，负责账号、代理、流量、子域名/端口和发布入口。

frp / frpc / frps	开源内网穿透工具及客户端/服务端；B 端运行 frpc，S 端运行 frps。
KB 模式	OpenCode 的知识库模式，限制文件访问、终端执行和工具能力范围。
API 分享	B 端通过 BearFRP 分享 LLM-Wiki API，而不是分享完整 OpenCode 页面。
远程知识库	C 端保存的 B 端 LLM-Wiki API 地址、Token、项目选择和健康状态配置。
SRS	Software Requirements Specification，软件需求规格说明书。
FR	Functional Requirement，功能性需求。
NFR	Non-Functional Requirement，非功能性需求。

1.6 参考资料

1. GB/T 8567-88 《计算机软件开发文件编制指南》。
2. 课程提供的《软件需求说明书（GB856T——88）》模板。
3. 需求规格说明书参考模板。
4. WikiBridge 计划任务书。
5. WikiBridge 软件设计说明书。
6. 新旧架构对比说明。
7. WikiBridge Desktop 交接说明。
8. WikiBridge 部署说明。
9. OpenCode KB 模式说明。
10. LLM-Wiki MCP server 说明。
11. 测试命令与脚本输出整理。
12. Playwright 人工测试截图说明。

2、软件的一般性描述

2.1 软件产品与其环境之间的关系

WikiBridge 不是单一 Web 站点，而是由 B 端本地知识库服务、S 端公网转发控制面和 C 端本地消费端共同组成的分布式原型系统。

系统组成部分	所在环境	与外部环境的关系
LLM-Wiki API	B 端	读取 B 端本地 Wiki 项目和文件，对外提供健康检查、项目、文件、搜索等 API。
frpc	B 端	连接 S 端 frps，把本地 LLM-Wiki API 发布为公网 URL。
BearFRP back-end	S 端	提供用户、代理、端口/子域名、脚本生成和状态管理 API。
frps / HTTP vhost / TCP 入口	S 端	接收 B 端 frpc 连接，并把 C 端请求转发到 B 端 LLM-Wiki API。
WikiBridge Desktop	B 端 / C 端	提供本地操作界面，覆盖发布 API、添加远程知识库、启动 OpenCode 和注册 MCP 等任务。
OpenCode	C 端	本地运行，使用 C 端模型配置生成回答，并通过 MCP 工具访问远程知识库。
llm-wiki MCP server	C 端	将 OpenCode 工具调用转换为远程 LLM-Wiki API 请求。
模型供应商 API	C 端外部依赖	C 端用户自行配置模型供应商、模型名和 API Key；B 端和 S 端不得代持该配置。
Docker Compose / CI / 测试脚本	开发与演示环境	支撑课程阶段部署、联调、黑盒检查和回归测试。

2.2 三端角色与职责

角色	主要职责	不承担的职责
B 端知识拥有者	导入本地资料，构建 Wiki，确认 LLM-Wiki API 可用，创建 BearFRP 发布入口，向 C 端分享 API URL。	不向外部访客暴露完整 OpenCode，不保存 C 端模型 API Key，不运行 C 端访客的 Agent 行为。

S 端管理员	维护 BearFRP、frps、端口/子域 名、账号、隧道状态、基础鉴权 和异常回收。	不保存原始资料、Wiki 正文、 向量索引、问答上下文或 C 端 模型配置。
C 端知识消费者	添加 B 端分享的远程知识库 API，本地保存模型配置，启动 OpenCode，注册 MCP 并进行 对话式知识消费。	不获得 B 端本机权限，不访 问 B 端未分享文件，不修改 B 端知识库内容。
项目开发者	实现与维护 LLM-Wiki、Bear- FRP、Desktop、OpenCode KB 模式、MCP server、部署脚本和 测试脚本。	不在需求文档中替代用户做 未确认的功能扩展承诺。

2.3 假设与前提条件

1. B 端用户已准备待构建的本地资料，并同意由 LLM-Wiki 在本机生成 Wiki 项目、
页面和索引。
2. B 端运行环境允许启动 LLM-Wiki API 和 frpc；S 端公网服务器可运行 BearFRP
backend 与 frps。
3. C 端用户可运行 WikiBridge Desktop、OpenCode 和 MCP server，并自行持有模
型供应商 API Key。
4. B 端与 C 端之间的访问依赖 S 端 frp 隧道；HTTP 子域名模式依赖真实可解析通
配域名，无法满足时可使用 TCP 端口模式。
5. 当前课程阶段以端到端原型和可验证链路为目标，不承诺商业级多租户 SLA、长
期运营、真实支付、完整企业权限体系和移动端原生应用。
6. 当前课程阶段以 2026 年 6 月 26 日形成的 V1.0 文档、测试报告和截图证据作为
验收基线。

2.4 产品范围边界

2.4.1 范围内

- 本地资料导入、Wiki 构建和项目/文件维护。
- LLM-Wiki API 健康检查、项目列表、文件列表、文件内容读取、搜索和必要元数
据能力。
- BearFRP 用户、代理、子域名/端口、frpc 脚本和隧道状态管理。
- B 端通过 BearFRP 发布 LLM-Wiki API。

- C 端添加远程知识库 API、检测健康状态、选择项目和保存 Token。
- C 端本地启动 OpenCode，并注册 llm_wiki_* MCP 工具。
- OpenCode KB 模式下对远程知识库进行只读、受控、对话式访问。
- Docker Compose 演示、健康检查、测试脚本和验收截图证据。

2.4.2 范围外

- 面向全网公开的博客、CDN 分发、搜索引擎收录和长期内容托管。
- S 端云端保存用户原始资料、Wiki 正文、向量库或问答上下文。
- 将 B 端完整 OpenCode、终端、文件系统或任意 Agent 工具暴露到公网。
- 真实支付、商业订阅、企业级权限审计、内容审核平台和大规模多租户运营。
- 移动端原生 App。
- 对任意第三方 MCP 工具的公网开放。

2.5 架构需求演进

方案	描述	需求结论
旧方案:B 端发布完整 OpenCode	B 端运行 LLM-Wiki 和 OpenCode, 通过 frp 将完整 OpenCode 页面发布给 C 端访问。	淘汰。该方案会混淆模型配置归属, 使 C 端访客的对话行为运行在 B 端 OpenCode 环境中, 并扩大 B 端文件访问和工具调用风险。
当前方案:C 端本地 OpenCode + MCP	B 端只发布 LLM-Wiki API; C 端本地启动 OpenCode, 并通过本地 MCP server 调用 B 端远程知识库 API。	作为本文档需求基线。B 端负责知识库生成与 API 分享, S 端负责公网转发控制面, C 端负责模型配置、Agent 和 MCP 消费。

3、软件功能需求描述

3.1 软件需求的用例模型

系统涉及 B 端知识拥有者、C 端知识消费者、S 端管理员和项目开发者四类参与者。顶层用例图如下：

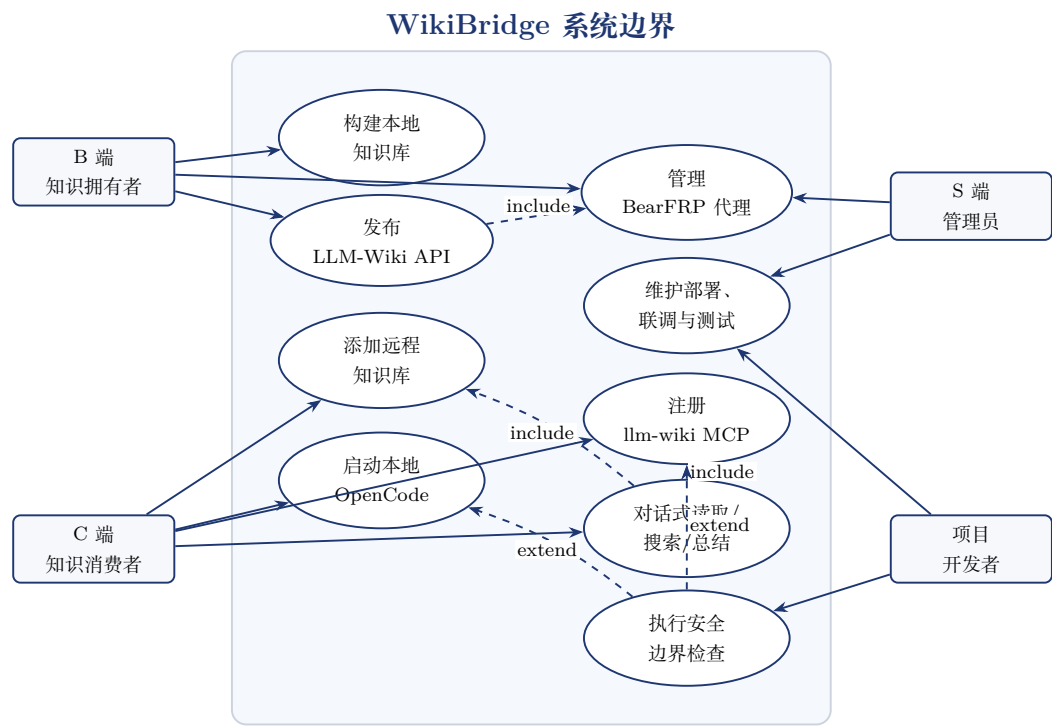


图 1: WikiBridge 用户用例图

3.2 关键业务流程

需求层主流程只描述用户可观察业务链路，不展开内部类、线程和数据库表。

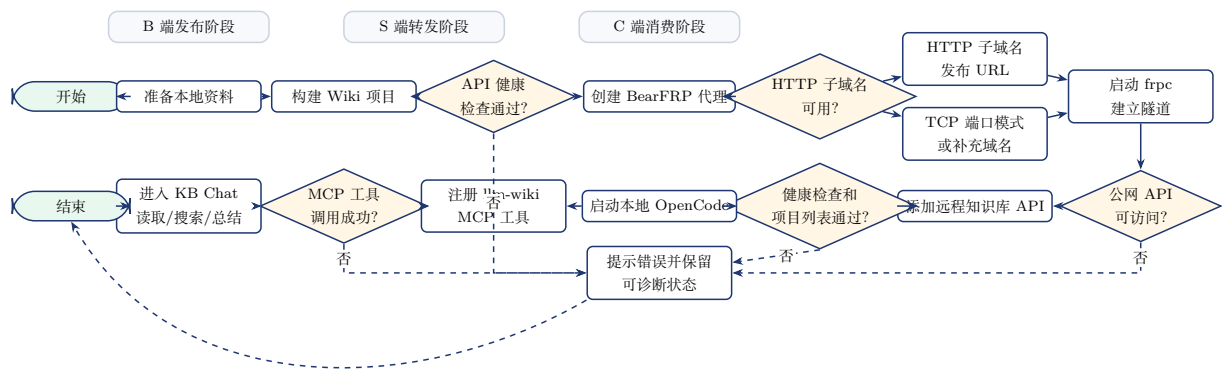


图 2: B 端 API 发布与 C 端 OpenCode/MCP 消费流程图

3.3 核心用例表

用例编号	用例名称	主要参与者	前置条件	成功结果	关联需求
------	------	-------	------	------	------

UC-01	构建本地知识库	B 端知识拥有者	B 端资料目录可访问, LLM-Wiki 可启动。	本地生成可被 API 查询的 Wiki 项目和文件。	FR-01、FR-02
UC-02	发布 LLM-Wiki API	B 端知识拥有者	本地 LLM-Wiki API 健康检查通过, BearFRP 可用。	获得 C 端可访问的公网 API URL。	FR-03、FR-04
UC-03	管理 BearFRP	B 端知识拥有者、S 端管理员	BearFRP back-end 和 frps 已运行。	代理可创建、查询、禁用并展示在线状态。	FR-04
UC-04	添加远程知识库	C 端知识消费者	C 端可访问 B 端分享 URL, 必要时持有 Token。	C 端保存远程知识库并选择项目。	FR-05
UC-05	启动本地 OpenCode	C 端知识消费者	OpenCode side-car 可用, 模型配置已保存。	本地 OpenCode 以 KB 模式运行。	FR-06、FR-09
UC-06	注册 llm-wiki MCP	C 端知识消费者	远程知识库可用, MCP server 可运行。	OpenCode 可用 llm_wiki_* 工具。	FR-07、FR-09
UC-07	对话式读取/搜索/总结知识库	C 端知识消费者	远程知识库、OpenCode 和 MCP 均连接成功。	用户获得基于远程知识库内容的回答。	FR-08
UC-08	维护部署、联调与测试	项目开发者、S 端管理员、测试人员	部署脚本、测试脚本和演示数据可用。	形成可复现命令输出、截图和验收记录。	FR-10

3.4 功能需求概览

编号	名称	优先级	主要参与者	对应用例
FR-01	本地资料导入与 Wiki 构建	P0	B 端知识拥有者	构建本地知识库

FR-02	LLM-Wiki API 健康检查、项目、文件、搜索能力	P0	B 端知识拥有者、C 端知识消费者	构建本地知识库、添加远程知识库、对话式消费
FR-03	B 端通过 BearFRP 发布 LLM-Wiki API	P0	B 端知识拥有者	发布 LLM-Wiki API
FR-04	S 端代理、隧道、端口/子域名和状态管理	P0	S 端管理员、B 端知识拥有者	管理 BearFRP 代理
FR-05	C 端添加远程知识库 API	P0	C 端知识消费者	添加远程知识库
FR-06	C 端本地启动 OpenCode	P0	C 端知识消费者	启动本地 Open-Code
FR-07	注册并调用 llm_wiki_* MCP 工具	P0	C 端知识消费者	注册 llm-wiki MCP
FR-08	对话式读取、搜索和总结远程知识库	P0	C 端知识消费者	对话式读取/搜索/总结知识库
FR-09	KB 模式安全边界	P0	B 端知识拥有者、C 端知识消费者、项目开发者	执行安全边界检查
FR-10	部署、健康检查、联调和测试支撑	P1	项目开发者、S 端管理员	维护部署、联调与测试

3.5 软件需求的分析模型与功能需求详述

3.5.1 FR-01 本地资料导入与 Wiki 构建

项目	内容
需求描述	系统应支持 B 端用户把本地资料导入 LLM-Wiki，并生成可被 API 查询的 Wiki 项目、页面和索引。
参与者	B 端知识拥有者。
前置条件	B 端资料目录存在；LLM-Wiki 可启动；用户具有本机资料访问权限。

基本流程	用户选择或准备本地资料目录；触发 Wiki 构建或刷新；LLM-Wiki 生成项目记录、Wiki 页面、文件列表和必要索引；用户在本地可浏览构建结果；系统通过 API 暴露项目列表和文件读取能力。
异常流程	资料路径不存在或无权限时，系统提示路径不可访问并停止构建；构建失败时保留错误日志和可重试入口；模型或构建配置缺失时，系统提示补充配置或使用已有静态资料降级验证。
后置条件	B 端本地存在可被 LLM-Wiki API 识别的项目、Wiki 文件和索引。
验收标准	至少一个演示项目可通过项目列表接口查询；至少一个 Mark-down 文件可通过文件内容接口读取；构建失败时不破坏已有项目数据。
关联证据	测试命令与脚本输出整理中的 sample-wiki fixture 准备；T-02/T-07 黑盒项目级检查。

3.5.2 FR-02 LLM-Wiki API 健康检查、项目、文件、搜索能力

项目	内容
需求描述	LLM-Wiki 应提供供 B 端发布和 C 端消费的受控 HTTP API，至少覆盖健康检查、项目列表、文件列表、文件内容读取和搜索能力。
参与者	B 端知识拥有者、C 端知识消费者、llm-wiki MCP server。
前置条件	LLM-Wiki API 已启动；目标项目已构建；如启用 Token，则调用方持有有效 Token。
基本流程	调用健康检查接口确认服务可用；调用项目列表接口获取可访问项目；调用文件列表接口获取项目内 Wiki 文件；调用文件内容接口读取允许路径内的文本内容；调用搜索接口按关键词或语义检索项目内容。
异常流程	API 未启动时返回连接失败或健康检查失败；Token 缺失或错误时返回 401/403；项目不存在、文件不存在或路径越权时返回明确错误，不泄露本机其它路径；搜索无结果时返回空结果和友好提示，不视为系统错误。
后置条件	C 端可基于 API 响应展示项目状态、文件内容和搜索结果。

验收标准	/api/v1/health、项目列表、项目文件、文件内容和搜索请求在演示数据下返回成功状态；非法 Token 和非法路径被拒绝；API 错误包含可诊断信息但不包含敏感信息。
关联证据	T-02 LLM Wiki API、T-07 end-to-end access、T-07 file content/search screenshots。

3.5.3 FR-03 B 端通过 BearFRP 发布 LLM-Wiki API

项目	内容
需求描述	系统应允许 B 端用户通过 BearFRP 把本地 LLM-Wiki API 发布为 C 端可访问的公网 API URL。
参与者	B 端知识拥有者。
前置条件	B 端 LLM-Wiki API 健康检查通过；B 端可连接 S 端 frps；BearFRP 账号和代理创建能力可用。
基本流程	用户在 B 端确认本地 LLM-Wiki API 地址和端口；登录或连接 BearFRP 控制面；选择 HTTP 子域名模式或 TCP 端口模式；系统创建代理并生成 frpc 配置或启动脚本；B 端启动 frpc 建立隧道；系统返回可分享的公网 LLM-Wiki API URL。
异常流程	本地 API 不可达时禁止发布并提示先修复 LLM-Wiki；HTTP 通配域名未配置时提示切换 TCP 模式或补充域名；端口冲突、余额不足、配额不足或认证失败时，控制面返回明确原因；frpc 断开时，系统显示离线状态并允许用户重试。
后置条件	C 端可通过公网 URL 访问 B 端 LLM-Wiki API；B 端可关闭或回收该入口。
验收标准	公网 URL 的健康检查和项目列表可访问；发布入口指向 LLM-Wiki API，不指向 B 端完整 OpenCode；关闭代理后公网入口不可继续访问。
关联证据	T-07 BearFRP published entry；T-06 auto-publish sidecar；桌面端联调流程。

3.5.4 FR-04 S 端代理、隧道、端口/子域名和状态管理

项目	内容
----	----

需求描述	S 端应提供 BearFRP 控制面和 frps 转发能力，用于管理用户、代理、子域名/端口、frpc 配置、在线状态和基础资源约束。
参与者	S 端管理员、B 端知识拥有者。
前置条件	S 端公网服务器可运行 BearFRP backend、frps 和控制面数据库；端口和域名配置已准备。
基本流程	管理员启动 BearFRP backend 和 frps；B 端用户创建 HTTP 或 TCP 代理；系统分配子域名或公网端口，生成 frpc 连接参数；frps 接收 frpc 连接并转发 C 端请求；控制面展示代理在线状态、连接时间、流量或连接数量；管理员可禁用异常代理或调整端口/子域名配置。
异常流程	子域名已占用或端口冲突时，系统拒绝创建并提示更换；frps token 不匹配时，连接被拒绝并记录状态；代理离线或心跳超时，控制面标记离线；资源超过限制时，系统应限流、拒绝新连接或提示升级配置。
后置条件	S 端只保存账号、代理和隧道状态，不保存知识正文和问答上下文。
验收标准	BearFRP API 可创建和查询代理；在线状态可观测；HTTP/TCP 模式至少一种可完成演示；管理员可停止异常代理。
关联证据	T-04 BearFRP user/proxy pytest；T-05 frps plugin and poller pytest；BearFRP 发布说明。

3.5.5 FR-05 C 端添加远程知识库 API

项目	内容
需求描述	C 端用户应能在 WikiBridge Desktop 中添加 B 端分享的 LLM-Wiki API 地址，并完成健康检查、项目读取、Token 保存和项目选择。
参与者	C 端知识消费者。
前置条件	C 端可访问 B 端分享的公网 API URL；如 API 开启鉴权，用户持有 Token。
基本流程	用户在“添加远程知识库”界面输入 API URL、名称和可选 Token；系统规范化 URL，并调用健康检查接口；系统调用项目列表接口；用户选择一个项目作为当前知识库；系统在 C 端本地保存远程知识库配置和最近检测状态。

异常流程	URL 格式错误时，界面提示修正；网络超时、DNS 失败或隧道离线时，系统提示远程不可达；401/403 时提示检查 Token；项目列表为空时，提示 B 端先构建或选择可分享项目。
后置条件	C 端远程知识库列表中出现该 API，并显示健康状态、当前项目和连接入口。
验收标准	C 端可保存、检测、切换和删除远程知识库；错误状态不会污染已有可用配置；Token 不以明文完整形式展示在日志和界面中。
关联证据	T-07 local entry OpenCode KB home; T-09 desktop project dashboard/compile/link screenshots。

3.5.6 FR-06 C 端本地启动 OpenCode

项目	内容
需求描述	C 端应在本机启动 OpenCode，并以 KB 模式作为远程知识库对话入口。
参与者	C 端知识消费者。
前置条件	C 端已安装或打包 OpenCode sidecar；用户已保存模型供应商、模型名和 API Key；至少一个远程知识库可用。
基本流程	用户保存或确认 C 端模型配置；点击连接远程知识库或启动 OpenCode；Desktop 启动本地 OpenCode 进程；OpenCode 以 KB 模式提供本地 Web 入口和健康检查；用户进入 KB Chat Session。
异常流程	OpenCode 二进制缺失或版本过旧时提示重新准备 sidecar；端口被占用时提示更换端口或停止旧进程；模型配置缺失时禁止进入正式问答并提示补充配置；OpenCode 启动后健康检查失败时，显示日志入口和重试按钮。
后置条件	C 端本地 OpenCode 可访问，页面标识 KB 模式，模型配置保存在 C 端。
验收标准	本地 OpenCode 健康检查成功；页面含 KB 模式标记；C 端模型 API Key 不上传至 B 端或 S 端。
关联证据	T-03 KB mode meta and UI; T-07 local entry OpenCode KB home; OpenCode KB 模式说明。

3.5.7 FR-07 注册并调用 llm-wiki MCP 工具

项目	内容
需求描述	C 端应把远程 LLM-Wiki API 注册为 OpenCode 可调用的 MCP 工具，并限制在 llm_wiki_* 知识库工具范围内。
参与者	C 端知识消费者、OpenCode、llm-wiki MCP server。
前置条件	C 端 OpenCode 已启动；远程知识库配置可用；MCP server 可运行；如远程 API 需要 Token，Token 已保存。
基本流程	Desktop 根据远程知识库生成 MCP server 配置；系统通过环境变量传入 API base URL 和 Token；OpenCode 注册 llm-wiki MCP；MCP server 暴露状态、项目、文件、读取、搜索和图谱等允许工具；OpenCode 在对话中调用 llm_wiki_status、llm_wiki_projects、llm_wiki_read_file、llm_wiki_search 等工具。
异常流程	MCP server 启动失败时提示检查 Node 版本、入口文件和依赖；远程 API 不可达时工具返回可读错误和 Desktop 修复提示；非 JSON 响应、API 错误和网络错误应被包装为可诊断工具错误；非白名单 MCP 或本地命令型 MCP 不得在 KB 模式下被任意添加。
后置条件	OpenCode 可以通过 MCP 访问远程知识库，但不能越过 LLM-Wiki API 直接扫描 B 端文件系统。
验收标准	MCP 测试通过；工具调用携带必要 Token；文件读取和搜索结果来自远程 LLM-Wiki API；非法 MCP 添加或危险工具调用被拒绝。
关联证据	T-02 LLM Wiki MCP tests；LLM-Wiki MCP server 说明；T-03 KB permission tests。

3.5.8 FR-08 对话式读取、搜索和总结远程知识库

项目	内容
需求描述	C 端用户应能在本地 OpenCode 中用自然语言读取、搜索和总结 B 端分享的远程知识库。
参与者	C 端知识消费者。
前置条件	FR-05、FR-06、FR-07 已完成；C 端模型配置有效；远程知识库中存在可读内容。

基本流程	用户在 KB Chat 中输入问题, 例如要求读取某个 Markdown 文件或搜索关键词; OpenCode 根据问题选择 llm_wiki_* 工具; MCP server 请求远程 LLM-Wiki API; LLM-Wiki 返回文件内容、搜索结果或项目元数据; OpenCode 使用 C 端模型配置生成回答、摘要或引用说明。
异常流程	未选择项目时, 系统提示先选择远程知识库项目; 文件不存在或路径不允许时, 工具返回明确错误; 内容过长时, 工具可返回截断标记并提示分段读取; 模型调用失败时, 保留工具结果并提示用户检查 C 端模型配置。
后置条件	用户获得基于远程知识库内容的回答; B 端只暴露受控 API 响应。
验收标准	C 端能完成至少一次文件读取、一次搜索和一次总结演示; 回答不得要求 B 端执行终端命令或访问未分享路径。
关联证据	T-07 llm-wiki-file-content; T-07 llm-wiki-search-results; T-09 desktop local wiki reader。

3.5.9 FR-09 KB 模式安全边界

项目	内容
需求描述	系统必须在需求层明确 KB 模式安全边界: B 端不得暴露完整 OpenCode; C 端 OpenCode 只能通过受控 llm_wiki_* MCP 工具访问远程知识库; 终端、本地命令、越权文件访问和非白名单 MCP 应被阻断。
参与者	B 端知识拥有者、C 端知识消费者、项目开发者、测试人员。
前置条件	C 端 OpenCode 以 KB 模式运行; 远程知识库通过 LLM-Wiki API 接入; 安全测试脚本可执行。
基本流程	系统在 OpenCode 页面注入 KB 模式标识; 文件访问限制在允许的知识库路径或远程 API 结果内; 公开 Wiki 或远程知识库内容按只读方式消费; Shell、PTY、终端、配置修改、动态 MCP 添加和危险权限在 KB 模式下被拒绝或隐藏; 日志隐藏 Token、API Key 和敏感路径。

异常流程	用户直接调用受限 API 时返回 403 或等价拒绝结果；用户尝试路径穿越、绝对路径、软链接逃逸或其它用户目录访问时被拒绝；旧 sidecar 或旧脚本试图发布 B 端 OpenCode 时，应被视为不符合当前需求基线。
后置条件	C 端可以对话式消费知识库，但不能获得 B 端 Agent、本地命令或越权文件访问能力。
验收标准	KB 模式 meta 可检测；终端/PTY/配置/MCP 动态添加等高危接口被拒绝；B 端发布入口指向 LLM-Wiki API；安全测试和 XSS 检查通过。
关联证据	T-03 KB mode meta and UI；T-08 XSS no alert；OpenCode KB 模式说明中的已验证项。

3.5.10 FR-10 部署、健康检查、联调和测试支撑

项目	内容
需求描述	项目应提供可复现的部署、健康检查、联调和测试支撑，使需求验收不依赖口头说明。
参与者	项目开发者、S 端管理员、测试人员。
前置条件	代码仓库、Docker、Node、Rust/Cargo、测试脚本和演示数据按文档准备；部分测试需要 Node 20.19+。
基本流程	使用 Docker Compose 或本地开发命令启动 LLM-Wiki、BearFRP、OpenCode 和必要网关；准备 sample-wiki 或等价演示数据；执行 LLM-Wiki API、BearFRP、OpenCode KB、MCP 和 Desktop 相关测试；记录命令输出、截图证据和剩余风险；将测试结果与 FR 编号关联。
异常流程	Node 版本不足时，脚本应提示所需运行时版本并终止对应自动化路径；Docker 或服务未启动时，黑盒检查应失败并说明不可达组件；真实 UI 截图采集失败时，应保留错误日志并重新执行截图脚本生成证据。
后置条件	项目组能够用命令、截图和报告证明核心链路可运行。
验收标准	测试脚本语法检查和空白检查通过；T-02/T-07 黑盒项目级检查通过；T-06 sidecar 检查通过；T-09 合同测试、Playwright 测试和截图采集通过。

关联证据 测试命令与脚本输出整理；Playwright 人工测试截图说明；测试脚本说明。

3.6 需求追踪矩阵

需求编号	关联用例	关联测试/证据	备注
FR-01	构建本地知识库	sample-wiki fixture; T-02/T-07	演示数据规模以测试报告记录为准。
FR-02	API 健康、项目、文件、搜索	T-02; T-07; 文件/搜索截图	API token 策略以实际配置为准。
FR-03	发布 LLM-Wiki API	T-06; T-07 BearFRP 发布入口	不再发布完整 Open-Code。
FR-04	BearFRP 代理管理	T-04; T-05; 部署说明	HTTP 子域名依赖通配域名。
FR-05	添加远程知识库	T-09 Desktop 截图; 桌面端联调流程	Desktop 截图编号见第 5 章。
FR-06	启动本地 Open-Code	T-03; T-07 local entry	Node/sidecar 版本需满足打包要求。
FR-07	MCP 注册和工具调用	T-02 MCP tests; T-03	工具命名采用 llm_wiki_*。
FR-08	对话式消费	T-07 文件/搜索截图; T-09 local wiki reader	模型调用效果受 C 端模型配置影响。
FR-09	KB 安全边界	T-03; T-08; KB 模式说明	旧脚本只作为历史资料，不作为当前需求。
FR-10	部署与测试支撑	测试命令输出; Playwright 截图说明	Node 22 补跑结果和截图证据已纳入测试报告。

4、其它软件需求描述

4.1 性能要求

编号	要求
NFR-PERF-01	在课程演示数据规模下，LLM-Wiki 健康检查、项目列表和文件列表应在可交互时间内返回；本地 P95 响应时间目标不超过 2 秒。
NFR-PERF-02	通过 BearFRP/frp 访问远程 LLM-Wiki API 时，系统应尽量保持问答前工具调用可用；网络抖动时应返回超时提示，不应导致 Desktop 或 OpenCode 无响应。
NFR-PERF-03	C 端启动 OpenCode 和注册 MCP 的过程应给出进度或状态反馈；演示环境启动时间目标不超过 30 秒。
NFR-PERF-04	搜索和文件读取结果过大时，API 或 MCP 工具应支持截断、分页或限制返回，避免单次工具调用阻塞对话。

4.2 安全与隐私要求

编号	要求
NFR-SEC-01	B 端对外只发布 LLM-Wiki API，不发布完整 OpenCode 页面、终端、文件系统或 Agent 执行环境。
NFR-SEC-02	S 端只保存账号、代理、端口/子域名、隧道状态等控制面数据，不保存原始资料、Wiki 正文、向量索引和问答上下文。
NFR-SEC-03	C 端模型供应商、模型名和 API Key 只保存在 C 端本地，不上传给 B 端或 S 端。
NFR-SEC-04	Token、API Key、密码和 frps/frpc 鉴权信息不得以明文完整形式出现在日志、截图和普通界面中。
NFR-SEC-05	KB 模式必须拒绝终端执行、PTY 创建或连接、越权文件读取、配置修改和非白名单 MCP 动态添加。
NFR-SEC-06	文件读取必须通过 LLM-Wiki API 或 KB 模式允许路径进行，路径穿越、绝对路径越权和软链接逃逸应被拒绝。
NFR-SEC-07	B 端发布入口应支持主动关闭、过期或管理员回收。滥用、异常流量或 Token 泄露时应能够停止对应代理。
NFR-SEC-08	展示远程知识库内容时应防止脚本执行和弹窗注入；XSS 检查结果应纳入验收证据。

4.3 可用性与可靠性要求

编号	要求
NFR-REL-01	LLM-Wiki API、BearFRP 代理、frp 隧道、远程知识库、Open-Code 和 MCP 注册状态应在界面或日志中可观测。
NFR-REL-02	任一组件不可达时，系统应指明不可达组件和建议修复动作，例如检查 Token、端口、域名、隧道或 sidecar 版本。
NFR-REL-03	发布入口、远程知识库配置和模型配置应在可恢复范围内本地持久化；失败重试不得删除已有可用配置。
NFR-REL-04	黑盒检查和截图脚本应在环境不满足时给出明确版本或依赖提示，不应把环境限制误判为功能失败。

4.4 兼容性与运行环境要求

编号	要求
NFR-COMP-01	WikiBridge Desktop 原型应面向主流桌面环境，课程验收阶段以 Linux 桌面环境和浏览器自动化截图为主要验证平台。
NFR-COMP-02	OpenCode、MCP server 和 Desktop 前端相关测试需要 Node 20.19+ 或 CI 中等价运行时；最终补跑环境采用 Node 22.13.1。
NFR-COMP-03	Docker Compose 演示环境应能启动 LLM-Wiki、BearFRP、OpenCode、nginx 和可选 frpc sidecar。
NFR-COMP-04	HTTP 子域名发布依赖公网域名和通配解析；未具备该条件时，系统应支持 TCP 端口模式完成演示。

4.5 界面要求

界面	需求
本地知识库/项目界面	展示本地项目、构建状态、文件或编译就绪状态；构建失败时给出可读错误。
BearFRP 发布界面	展示本地 API 地址、代理类型、子域名/端口、隧道状态、公网 URL 和关闭入口。

远程知识库界面	支持输入 API URL、名称、Token、健康检查、项目选择、连接、删除和错误提示。
模型配置界面	支持 C 端填写模型供应商、模型名、API Key，并说明该配置保存在 C 端。
OpenCode 入口界面	展示本地 OpenCode 启动状态、KB 模式状态、进入对话入口和运行日志。
MCP 连接界面	展示 MCP 注册状态、允许工具列表和失败原因。
错误与日志界面	错误信息应面向用户可理解，不只暴露底层堆栈；敏感信息应脱敏。

4.6 进度要求

项目	要求
课程阶段主链路	优先完成 B 端 LLM-Wiki API 发布、C 端远程知识库添加、OpenCode 启动、MCP 注册和对话式读取/搜索。
文档交付	需求规格说明书、软件设计说明书、测试报告、计划任务书和个人总结材料应保持术语一致。
最终交付基线	2026 年 6 月 26 日形成 V1.0 需求、SDS V1.2 设计和 V1.0 测试材料；T-09 Desktop 截图与 Node 22 补跑结果纳入最终验收证据。

4.7 交付要求

类型	交付物
程序	WikiBridge Desktop、LLM-Wiki API、BearFRP/frps 集成、OpenCode KB 模式、llm-wiki MCP server、Docker Compose 和测试脚本。
文档	本需求规格说明书、计划任务书、软件设计说明书、测试文档/测试报告、部署说明、Desktop 交接说明和演示说明。
证据	测试命令输出、Playwright 截图、API 证据页、架构演进说明和最终答辩截图。

4.8 验收要求

1. 文档验收：本文档应覆盖 GB/T 8567-88 和课程参考 SRS 的核心章节，包括引言、一般性描述、功能需求、其它需求和软件原型。
2. 功能验收：FR-01 至 FR-10 每项均应具备参与者、前置条件、基本流程、异常流程和验收标准。
3. 链路验收：B 端构建 Wiki、发布 LLM-Wiki API，C 端添加远程知识库、本地启动 OpenCode、注册 MCP，并完成读取/搜索/总结演示。
4. 安全验收：不得通过公网暴露 B 端完整 OpenCode；C 端 KB 模式应拒绝终端、本地命令、越权文件和非白名单 MCP。
5. 测试验收：以测试命令与脚本输出整理、测试分析报告和 CI 结果为依据，记录通过、失败和环境差异。
6. 截图验收：第 5 章列出的截图证据已按测试项编号，可用于最终提交和答辩演示。

5、软件原型与演示证据

5.1 原型界面文字描述

当前 WikiBridge 原型以桌面端工作台和本地 OpenCode KB 页面为主要入口：

- Desktop 本地知识库界面：展示项目列表、构建状态、编译就绪状态和本地 Wiki 阅读入口。该界面用于证明 B 端可以准备和检查本地知识库。
- Desktop BearFRP/访问连接界面：展示发布 API、代理状态、公网 URL 和联调错误。该界面用于证明 B 端分享的是 LLM-Wiki API。
- Desktop 远程知识库界面：允许 C 端粘贴 B 端分享地址、填写 Token、执行健康检查、读取项目列表并连接远程知识库。
- Desktop OpenCode 面板：负责启动本地 OpenCode，显示运行状态，并进入 KB Chat。
- OpenCode KB 页面：页面带有 KB 模式标识，隐藏或拒绝终端、危险文件访问和动态 MCP 等能力，作为 C 端对话式消费入口。
- API 证据页：当真实 UI 截图受环境限制时，可使用同源 API 返回数据生成证据页，证明文件内容读取、搜索结果和安全检查通过。

5.2 已完成截图证据

测试项	截图文件	当前结果	证据来源	对应需求
T-07	本地入口 OpenCode KB 页面截图	PASS	real-ui	FR-06、FR-09
T-03	KB 模式页面与 meta 标识截图	PASS	real-ui	FR-06、FR-09
T-07	LLM-Wiki 知识库页面截图	PASS	api-evidence	FR-01、FR-02
T-07	LLM-Wiki 文件内容页面截图	PASS	api-evidence	FR-02、FR-08
T-07	LLM-Wiki 搜索结果页面截图	PASS	api-evidence	FR-02、FR-08
T-08	XSS 样例页面截图	PASS	api-evidence	FR-09
T-07	BearFRP 发布入口页面截图	PASS	real-ui	FR-03、FR-04
T-09	桌面端知识库项目仪表盘截图	PASS	real-ui	FR-01、FR-05
T-09	桌面端编译准备状态截图	PASS	real-ui	FR-01
T-09	桌面端访问连接状态截图	PASS	real-ui	FR-03、FR-05
T-09	桌面端消费端远程知识库入口截图	PASS	real-ui	FR-01、FR-08

5.3 演示脚本

1. B 端启动 LLM-Wiki，并确认 /api/v1/health 和项目列表可访问。
2. B 端通过 BearFRP 创建代理，获得公网 LLM-Wiki API URL。
3. C 端打开 WikiBridge Desktop，保存自己的模型配置。
4. C 端在远程知识库界面粘贴公网 API URL，执行健康检查并选择项目。
5. C 端连接该远程知识库，Desktop 启动本地 OpenCode 并注册 llm-wiki MCP。
6. C 端在 KB Chat 中提问，例如“读取知识库里的概览文件并总结主要内容”或“搜索 WikiBridge 相关内容”。
7. 演示安全边界：确认页面处于 KB 模式，终端/PTY/动态 MCP 等高危能力不可用，B 端没有向公网暴露完整 OpenCode。

第三部分

软件设计说明书

(终版)

文档变更历史记录

序号	变更日期	变更人员	变更内容详情描述	变更后的版本号
1	2026 年 6 月 1 日	项目组	启动文档编写，形成初始项目背景、范围和阶段性内容。	V0.1
2	2026 年 6 月 18 日	项目组	补充核心功能、架构、测试任务和阶段性实现材料。	V0.9
3	2026 年 6 月 26 日	项目组	根据最终联调结果整理终版文档并纳入分组实习报告，补充测试方法、缺陷矩阵和个人贡献评分表。	V1.0

文档编号: WikiBridge-SDS-1.2

WikiBridge

软件设计规格说明书

零代码本地知识库 API 分享与 OpenCode/MCP 消费平台

项目名称: WikiBridge

文档版本: V1.2

编写人员: 项目组

适用阶段: 课程设计原型与系统设计评审

日期: 2026 年 6 月 26 日

文档变更历史记录

序号	变更日期	变更人员	变更内容详情描述	版本
1	2026 年 6 月 1 日	项目组	启动系统设计调研，明确本地知识库、内网穿透、桌面端和对话式知识消费的总体设计目标。	V0.1
2	2026 年 6 月 10 日	项目组	补充 B 端、S 端、C 端初始职责划分，形成体系结构、用户界面、用例和部署设计草案。	V0.5
3	2026 年 6 月 14 日	项目组	基于现有项目简介、汇报材料、架构图与用例图，撰写 WikiBridge 软件设计规格说明书主体内容，形成设计说明书 V1.0。	V1.0
4	2026 年 6 月 25 日	项目组	根据联调结果完成 MCP 架构调整：淘汰 B 端发布完整 OpenCode 方案，改为 B 端仅发布 LLM-Wiki API, S 端提供 BearFRP/frps 控制面，C 端本地运行 OpenCode 并通过 MCP 消费远程知识库。	V1.1
5	2026 年 6 月 26 日	项目组	整理章节、图表、术语和参考资料，同步需求、计划和测试材料口径，按课程提交格式生成软件设计说明书终版。	V1.2

目录

文档变更历史记录	I
1 引言	1
1.1 编写目的	1
1.2 读者对象	1
1.3 软件项目概述	1
1.4 文档概述	2
1.5 定义	2
1.6 参考资料	3
2 软件设计约束	3
2.1 软件设计目标和原则	3
2.2 软件设计的约束和限制	4
3 软件设计	4
3.1 软件体系结构设计	4
3.1.1 B 端子系统	5
3.1.2 S 端子系统	5
3.1.3 C 端子系统	5
3.2 用户界面设计	5
3.3 用例设计	6
3.3.1 构建本地知识库用例	7
3.3.2 发布 LLM-Wiki API 用例	7
3.3.3 添加远程知识库用例	7
3.3.4 启动本地 OpenCode 与注册 MCP 用例	7
3.3.5 对话式读取与搜索用例	7
3.4 核心流程设计	8
3.5 类设计和构件设计	8
3.5.1 关键类关系	9
3.5.2 关键方法设计	9
3.6 数据设计	10

3.6.1	B 端数据结构	10
3.6.2	C 端本地配置数据	11
3.6.3	S 端控制面数据表	11
3.6.4	数据安全设计	12
3.7	部署设计	12
3.7.1	B 端部署	12
3.7.2	S 端部署	12
3.7.3	C 端部署	13
3.8	异常处理与可靠性设计	13
3.9	测试设计	13
结论		14

1、引言

1.1 编写目的

软件设计过程包括体系结构设计、用户界面设计、用例设计、构件设计、类设计、数据设计和部署设计。本文档以软件设计规格说明书的形式，对 WikiBridge 零代码本地知识库 API 分享与 OpenCode/MCP 消费平台的设计方案进行系统描述，形成可用于评审、开发分工、测试设计和后续迭代的设计基线。

本文档重点说明系统如何在 B 端生成和持有本地知识库，如何通过 BearFRP/frp 将 LLM-Wiki API 发布为受控公网入口，以及 C 端如何在本地运行 OpenCode 并通过 `llm-wiki-*` MCP 工具访问远程知识库。本文档以联调后确定的新架构为准：B 端不再直接发布完整 OpenCode 页面，也不把 C 端访客的 Agent 行为运行在 B 端环境中。

1.2 读者对象

本文档面向以下读者：

- 项目评审人员和课程设计指导教师，用于评估系统设计的完整性、合理性和可实现性。
- 软件设计人员和开发人员，用于理解系统边界、模块职责、关键流程和数据结构。
- 测试人员和质量保证人员，用于设计功能测试、异常测试和端到端验收场景。
- 配置管理人员和运维人员，用于理解本地端、服务器端和公网访问端的部署关系。

1.3 软件项目概述

项目名称为 WikiBridge，定位为零代码本地知识库 API 分享与受控 Agent 化消费平台。项目面向教师、学生、研究者、小团队和其他低技术门槛用户，解决个人资料分散、AI 知识库部署复杂、本地知识库难以安全分享、Agent 能力公网暴露边界不清的问题。

系统的核心思想是把知识库生成、发布和消费拆分到三个明确边界中。B 端知识库拥有者在本地运行 LLM-Wiki，保留原始资料、Wiki 页面和知识库 API；B 端只通过 frpc/frps 发布 LLM-Wiki API，而不是发布完整 OpenCode Agent。S 端运行 BearFRP 控制面和 frps，负责账号、鉴权、子域名或端口分配、隧道状态和公网入口转发。C 端知识消费者在自己的设备上运行 WikiBridge Desktop 与本地 OpenCode，保存自己的模型供应商、模型名和 API Key，并通过本地 MCP server 将 B 端分享的 LLM-Wiki API 注册为 `llm-wiki-*` 工具。

系统划分为三个端：

- B 端：知识库拥有者和发布者所在环境，负责本地资料整理、LLM-Wiki 构建、LLM-Wiki API 服务和 frpc 发布。

- S 端：公网服务器端，负责 BearFRP 控制面、frps、HTTP vhost、发布入口、账号鉴权和隧道状态。
- C 端：知识消费者所在环境，负责本地 OpenCode、MCP 注册、模型配置、远程知识库连接和对话式知识消费。

1.4 文档概述

本文档共分为三大部分。第一部分为引言，说明编写目的、读者对象、项目概述、术语定义和参考资料。第二部分为软件设计约束，说明设计目标、设计原则、运行约束、安全约束和技术约束。第三部分为软件设计，分别描述体系结构、用户界面、用例、类与构件、数据和部署。

1.5 定义

术语	定义
WikiBridge	本项目名称，表示连接本地知识库、受控公网入口和本地 Agent 消费端的桥接平台。
B 端	知识库所有者/发布者所在环境，保存本地资料、Wiki 内容和 LLM-Wiki API。
S 端	公网服务器端，运行 BearFRP 控制面、frps 和 HTTP vhost。
C 端	知识消费者所在环境，本地运行 WikiBridge Desktop、OpenCode 和 MCP server。
LLM-Wiki	本地知识库编译与 API 服务，提供健康检查、项目列表、文件读取和搜索等接口。
OpenCode	C 端本地对话式知识消费入口，负责模型调用、KB Chat Session 和工具调用。
MCP	Model Context Protocol，用于将远程 LLM-Wiki API 注册为 OpenCode 可调用工具。
BearFRP	本项目中的 frp 控制面，负责账号、代理、流量、子域名/端口和发布入口。
frp / frpc / frps	开源内网穿透工具及其客户端/服务端；B 端运行 frpc，S 端运行 frps。
KB 模式	OpenCode 的知识库模式，限制文件访问和工具能力范围。

API 分享	B 端通过 BearFRP 分享 LLM-Wiki API，而不是分享完整 OpenCode 页面。
控制面	S 端负责的账号、鉴权、隧道调度、状态记录和资源控制能力。
数据面	B 端知识内容、C 端模型配置、MCP 工具调用和远程 API 响应等业务数据流。

1.6 参考资料

1. WikiBridge 部署和 BearFRP 发布说明。
2. WikiBridge Desktop 交接说明。
3. 新旧架构对比说明。
4. 软件设计说明书图表素材。
5. 软件设计规格说明书模板：软件设计规格说明书-空巢老人看护系统.doc。

2、 软件设计约束

2.1 软件设计目标和原则

WikiBridge 的软件设计目标是在课程设计周期内形成可演示、可解释、可扩展的端到端原型设计。系统应让 B 端用户不需要手写 frp 配置即可发布本地 LLM-Wiki API，让 C 端用户只需粘贴远程知识库 API 地址、配置自己的模型 Key，即可在本地 OpenCode 中对远程知识库进行对话式消费。

系统设计遵循以下原则：

- 本地优先原则：B 端原始资料、Wiki 页面和 LLM-Wiki 数据保留在 B 端；C 端模型供应商、模型名和 API Key 保留在 C 端；S 端不保存知识正文和问答上下文。
- B/S/C 职责分离原则：B 端负责知识库生成和 API 分享，S 端负责 BearFRP 控制面和公网转发，C 端负责本地 OpenCode、MCP 注册和模型调用。
- Agent 边界收敛原则：不通过 frp 暴露 B 端完整 OpenCode；OpenCode Agent 能力运行在 C 端本地，且通过 KB 只读模式和 llm-wiki-* MCP 白名单收紧工具范围。
- 模块化与低耦合原则：LLM-Wiki API、BearFRP/frp、WikiBridge Desktop、Open-Code sidecar 和 MCP server 通过清晰接口协作，便于替换和扩展。

- 主动触发原则：发布 API、添加远程知识库、启动 OpenCode、注册 MCP 和连接远程知识库均由用户显式操作或确认。
- 可观测原则：LLM-Wiki 健康状态、frp 隧道状态、远程知识库可用性、OpenCode 启动状态和 MCP 注册状态均应在界面中可见。
- 可追踪原则：功能需求、用例、模块、数据结构和测试场景之间保持可追踪关系。

2.2 软件设计的约束和限制

约束类别	约束说明
运行环境	B 端运行 LLM-Wiki API 和 frpc；S 端运行 BearFRP back-end、frps 和控制面数据库；C 端运行 WikiBridge Desktop、OpenCode 和 MCP server。
网络条件	B 端通常处于内网或 NAT 环境，公网访问依赖 frpc 与 frps 建立隧道；HTTP 子域名模式依赖真实可解析的通配域名。
隐私边界	S 端不保存用户原始资料、Wiki 正文、向量数据或问答上下文；C 端模型 API Key 不上传到 S 端或 B 端。
安全约束	B 端不暴露完整 OpenCode；发布入口需要可回收、可鉴权、可限速；Token 和 API Key 不应明文长期暴露在界面或日志中。
能力约束	OpenCode KB 模式应阻断危险文件访问、终端执行和不受控工具能力；MCP 接入只允许项目内 llm-wiki-* 工具。
技术约束	当前阶段以原型验证为主，优先保证 B/S/C 主链路闭环；静态 Wiki 页面浏览可作为降级方案，不作为主消费路径。
课程周期	设计以可演示闭环为优先，先打通 B 端 API 发布、C 端远程知识库添加、OpenCode + MCP 对话，再逐步完善体验和边界场景。

3、 软件设计

3.1 软件体系结构设计

系统采用 B/S/C 三端分层架构。B 端负责知识库内容和 LLM-Wiki API，S 端负责 BearFRP 控制面和公网转发，C 端负责本地 OpenCode、MCP 注册和模型调用。总体架构如图 1 所示。

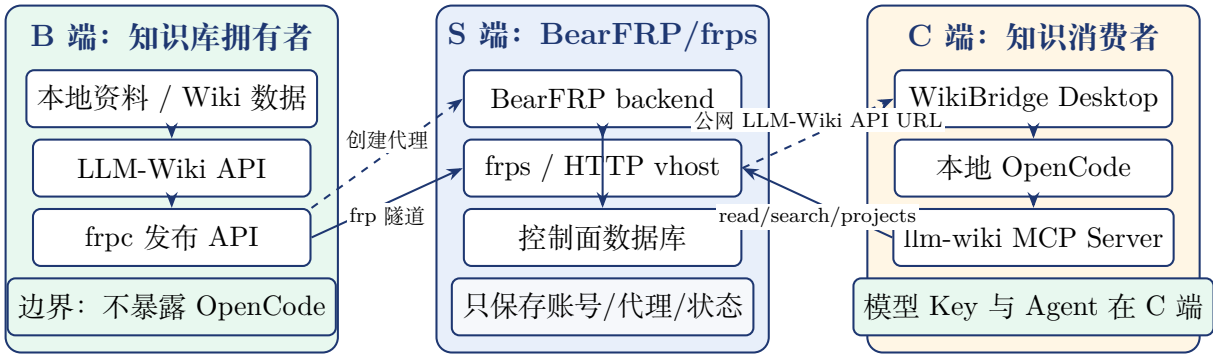


图 1: WikiBridge B/S/C 总体架构图

3.1.1 B 端子系统

B 端是知识库拥有者的核心操作环境，承担知识内容持有和 API 发布职责。其内部包含 LLM-Wiki 数据目录、LLM-Wiki API 服务、WikiBridge Desktop 发布控制界面和 frpc。B 端可以构建、浏览和管理本地知识库，但对外只发布 LLM-Wiki API，不发布完整 OpenCode 页面。

LLM-Wiki API 至少提供健康检查、项目列表、文件读取、搜索和元数据接口。frpc 仅将该 API 映射到 S 端公网入口，避免外部访客获得 B 端本机文件系统、模型 Key 或 OpenCode Agent 能力。

3.1.2 S 端子系统

S 端只处理平台控制面和转发面，不进入用户知识内容。BearFRP backend 负责用户、余额、代理、发布记录、脚本生成和在线状态；frps 负责接收 B 端 frpc 连接并根据 HTTP vhost 或 TCP 端口转发请求；控制面数据库保存账号、代理和隧道状态，不保存原始资料、Wiki 正文或问答上下文。

3.1.3 C 端子系统

C 端是知识消费者的本地 Agent 环境。WikiBridge Desktop 负责保存远程知识库 API 地址、检测健康状态、选择项目、启动本地 OpenCode 并注册 llm-wiki-* MCP。OpenCode 在 KB 模式下调用 MCP 工具读取或搜索远程知识库，模型供应商、模型名和 API Key 均保存在 C 端本地。

3.2 用户界面设计

WikiBridge Desktop 以 B 端发布和 C 端消费两类任务为核心，主要界面如表 4 所示。界面设计目标是让用户无需手写 frp 配置、MCP 配置和 OpenCode 启动命令，也能完成远程知识库发布和消费。

表 4: WikiBridge Desktop 主要界面设计

界面	主要元素	职责
欢迎/引导界面	项目简介、B/S/C 边界、本地优先说明、开始按钮	解释系统用途和隐私边界，引导用户进入工作台。
模型设置界面	模型供应商、模型名、API Key、保存/清除按钮	在 C 端本地保存模型配置，不上传到 S 端或 B 端。
本地 OpenCode 面板	OpenCode 地址、启动/进入对话/停止按钮、运行状态	启动 C 端本地 OpenCode, 展示健康状态和 KB Session 入口。
BearFRP 发布面板	本地 LLM-Wiki 地址、代理类型、子域名/端口、隧道状态	供 B 端创建和管理 LLM-Wiki API 公网入口。
远程知识库面板	API 地址、名称、Token、健康检查、项目列表	供 C 端添加 B 端分享的 LLM-Wiki API 并选择项目。
MCP 连接面板	MCP 名称、工具列表、连接/检测按钮、错误提示	注册 llm-wiki-* MCP 并检测 read_file、search 等工具。
远程知识库列表	已保存 API、可用状态、当前项目、连接按钮	展示已添加的远程知识库，并支持切换或删除。
系统日志与错误提示	OpenCode 日志、MCP 注册错误、frp 状态、远程 API 错误	让联调和验收时能定位端口、域名、配置和 API 错误。

界面跳转关系遵循“配置模型和服务状态、添加远程知识库、连接 MCP、进入 OpenCode 对话”的结构。发生 URL 不完整、通配域名未配置、远程 API 不可达、OpenCode 未就绪或 MCP 注册失败时，系统应在当前界面给出明确状态和可执行处理建议，而不是只输出底层错误日志。

3.3 用例设计

系统主要参与者包括 B 端知识拥有者、C 端知识消费者和 S 端管理员。用例关系如图 2 所示。

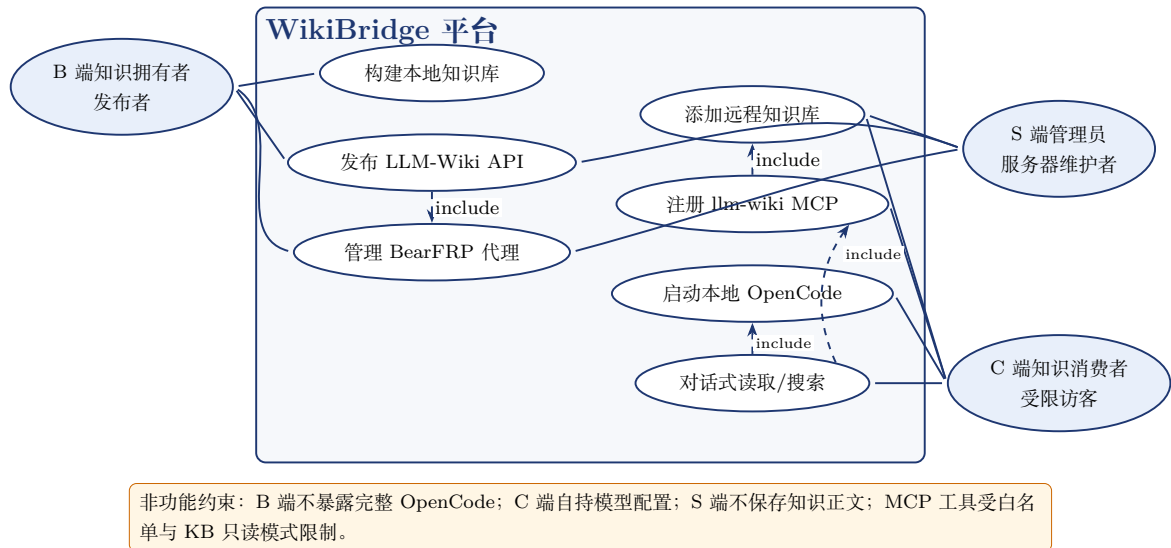


图 2: WikiBridge 用例图

3.3.1 构建本地知识库用例

B 端知识拥有者在本地准备资料并运行 LLM-Wiki，系统生成或维护本地 Wiki 项目、文件和索引。该用例由 LLM-Wiki 内部构建流程承担，WikiBridge Desktop 主要关注服务健康、项目列表和 API 是否可分享。

3.3.2 发布 LLM-Wiki API 用例

B 端用户登录 BearFRP 后创建 HTTP 或 TCP 代理。系统校验本地 LLM-Wiki API 可访问后启动 frpc，将本地 API 映射到 S 端公网入口。HTTP 子域名模式要求真实通配域名已经解析到 S 端；否则系统应提示配置域名或切换 TCP 模式。发布成功后，B 端获得可分享的 LLM-Wiki API URL。

3.3.3 添加远程知识库用例

C 端用户在 WikiBridge Desktop 中粘贴 B 端分享的 LLM-Wiki API URL，并可填写名称和 Token。系统调用健康检查和项目列表接口，确认 API 可达后保存远程知识库配置，并允许用户选择具体项目。

3.3.4 启动本地 OpenCode 与注册 MCP 用例

C 端用户保存模型配置后，Desktop 启动本地 OpenCode 并创建 KB Chat Session。随后系统注册 llm-wiki-* MCP，使 OpenCode 能调用 llm_wiki_read_file、llm_wiki_search 等受控工具访问远程 LLM-Wiki API。

3.3.5 对话式读取与搜索用例

C 端用户在本本地 OpenCode 中提问。OpenCode 根据模型推理和工具描述调用 llm-wiki MCP，MCP server 向远程 LLM-Wiki API 发起文件读取、项目列表或搜索请求。工具

结果返回 OpenCode 后，由 C 端模型配置生成回答。该过程中 B 端不运行 C 端访客的 Agent 行为。

3.4 核心流程设计

系统主流程由 B 端 API 发布、C 端远程知识库添加、OpenCode 启动、MCP 注册和工具调用组成，如图 3 所示。

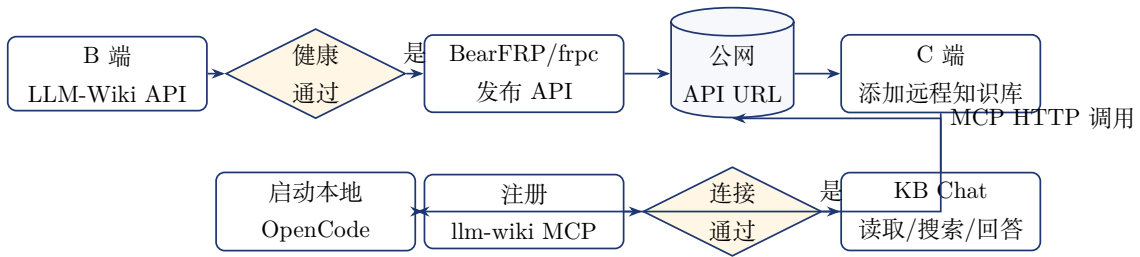


图 3: B 端 API 发布与 C 端 OpenCode/MCP 消费核心流程图

3.5 类设计和构件设计

当前阶段采用构件级设计结合关键类设计。构件划分如表 5 所示。

表 5: 主要构件职责

构件	关键类/对象	职责
桌面端壳层	DesktopShell	聚合 BearFRP、OpenCode、远程知识库和 MCP 状态，提供统一操作入口。
	StatusPanel	
	ErrorToast	
模型配置	ModelConfigStore	在 C 端本地保存模型供应商、模型名和 API Key，支持保存和清除。
	ProviderConfig	
	ApiKeyStore	
B 端发布控制	LlmWikiServiceRef	检查本地 LLM-Wiki API，创建 BearFRP 代理，启动和监控 frpc。
	FrpcController	
	TunnelStatusWatcher	
BearFRP 控制面	BearfrpClient	处理认证、代理创建、HTTP/TCP 发布入口、在线状态和脚本生成。
	ProxyService	
	FrpsStatusCollector	

构件	关键类/对象	职责
C 端远程知识库	RemoteKnowledgeBase	保存远程 API URL 和 Token，执行健康检查、项目列表、文件读取和搜索。
	LlmWikiApiClient	
	ProjectSelector	
OpenCode sidecar	OpenCodeSidecar	启动本地 OpenCode，创建 KB Chat Session，检测运行状态。
	KbSessionManager	
	OpenCodeHealthProbe	
MCP 注册与工具	McpRegistry	注册 llm-wiki-* MCP，限制工具名称、命令、入口文件和环境变量。
	LlmWikiMcpServer	
	ToolWhitelist	
KB 安全边界	KbGuard	在 OpenCode KB 模式下阻断危险文件访问、写入操作和不受控工具调用。
	ReadonlyPolicy	
	ToolPolicy	

3.5.1 关键类关系

系统类关系以端侧控制类协调任务、实体类保存状态、边界类展示界面为主。核心关系如下：

- 桌面端壳层聚合 OpenCode sidecar、远程知识库、MCP 注册和 frpc 控制器的状态，并把错误映射为用户可理解提示。
- 远程知识库对象持有远程 API URL、Token 和当前项目，依赖 LLM-Wiki API 客户端执行健康检查、项目列表、文件读取和搜索。
- OpenCodeSidecar 负责启动本地 OpenCode，并由 KbSessionManager 创建 KB Chat Session。
- MCP 注册器读取远程知识库配置，注册 llm-wiki MCP server，并通过工具白名单限制 llm-wiki-* 工具范围。
- frpc 控制器调用 BearFRP 客户端获取代理配置，再启动 B 端 frpc 进程；隧道状态监听器负责在线状态和重连提示。
- KbGuard 约束 OpenCode KB 模式下的文件访问、写入和工具调用策略。

3.5.2 关键方法设计

类	方法	说明
ModelConfigStore	<code>save(provider, model, apiKey)</code>	在 C 端安全保存模型配置，不上传到 S 端。
LlmWikiApiClient	<code>health(baseUrl); listProjects()</code>	检查远程 LLM-Wiki API 可用性并读取项目列表。
Remote KnowledgeBase	<code>connect(); selectProject(id)</code>	保存 API URL、Token 和当前项目，并维护可用状态。
OpenCodeSidecar	<code>start() / stop() / health()</code>	启动或停止本地 OpenCode，并返回运行状态。
KbSessionManager	<code>createKbSession(remoteKb)</code>	为远程知识库创建 OpenCode KB Chat Session。
McpRegistry	<code>registerLlmWiki(remoteKb)</code>	注册 llm-wiki-* MCP，并写入必要环境变量。
LlmWikiMcpServer	<code>readFile(project, path); search(query)</code>	将 OpenCode 工具调用转换为远程 LLM-Wiki API 请求。
FrpcController	<code>publishApi(localPort)</code>	B 端申请公网入口、生成配置、启动 frpc 并返回 API URL。
BearfrpClient	<code>createProxy(type, localPort)</code>	调用 BearFRP 控制面创建 HTTP/TCP 代理。
KbGuard	<code>enforceReadonly(toolCall)</code>	在 KB 模式下阻断写入、删除、终端执行和非白名单 MCP。

3.6 数据设计

WikiBridge 的数据设计分为 B 端知识库数据、S 端控制面数据和 C 端本地配置数据。系统明确避免把用户知识内容上传到 S 端持久化保存。

3.6.1 B 端数据结构

路径/文件	数据内容	说明
-------	------	----

<code>/raw</code>	原始资料	B 端用户本地保存的 Word、PDF、网页摘录、笔记、Markdown、TXT 等。
<code>/wiki</code>	Wiki 页面	LLM-Wiki 生成或维护的结构化页面和元数据。
<code>/index</code>	本地索引	供 LLM-Wiki 搜索和 API 查询使用的索引。
<code>llm-wiki config</code>	服务配置	B 端 LLM-Wiki 服务端口、项目目录、模型或构建配置。
<code>frpc config</code>	发布配置	B 端 frpc 连接 S 端 frps 的 token、协议、子域名或端口。
<code>publish cache</code>	发布状态缓存	当前公网 API URL、隧道状态、最近连接时间和可回收标识。

3.6.2 C 端本地配置数据

数据项	说明
<code>ModelConfig</code>	模型供应商、模型名和 API Key 保存位置。API Key 应使用系统安全存储能力或加密保存。
<code>Remote KnowledgeBase</code>	远程 LLM-Wiki API URL、名称、Token、当前项目、健康检查状态和最近检测时间。
<code>OpenCodeState</code>	本地 OpenCode 地址、端口、运行状态、当前 KB Session。
<code>McpRegistration</code>	MCP 名称、命令、入口文件、环境变量、工具列表和注册状态。
<code>UserPreference</code>	默认模型、默认远程知识库、日志级别和界面偏好。

3.6.3 S 端控制面数据表

S 端只保存平台控制面数据。建议设计以下数据表：

数据表	字段说明
<code>T_User</code>	<code>id</code> 、 <code>account</code> 、 <code>password_hash</code> 、 <code>display_name</code> 、 <code>status</code> 、 <code>created_at</code> 、 <code>last_login_at</code> 。用于保存平台账号，不保存知识内容。
<code>T_AccessToken</code>	<code>id</code> 、 <code>user_id</code> 、 <code>token_hash</code> 、 <code>expire_at</code> 、 <code>revoked_at</code> 。用于 Token 校验和吊销。

T_Proxy	id、user_id、type、subdomain、public_port、local_port、status、created_at、closed_at。用于记录 HTTP/TCP 发布入口。
T_TunnelStatus	id、proxy_id、online、bandwidth_in、bandwidth_out、connection_count、last_heartbeat。用于管理 frp 状态。
T_RateLimitPolicy	id、user_id、max_bandwidth、max_connections、enabled。用于基础限速和防滥用。

3.6.4 数据安全设计

本地配置中的 Token 和 API Key 应加密保存或使用系统安全存储能力。日志中不得输出完整 Token、API Key、用户密码、原始资料全文和模型请求完整上下文。S 端密码只保存哈希值，Token 只保存哈希或可验证摘要。发布入口应支持用户主动关闭，管理员也可在滥用或异常流量场景下回收入口。B 端不得将完整 OpenCode 暴露到公网，C 端不得将模型 API Key 上传给 B 端或 S 端。

3.7 部署设计

系统采用 B/S/C 分布式部署方式，部署关系如图 4 所示。

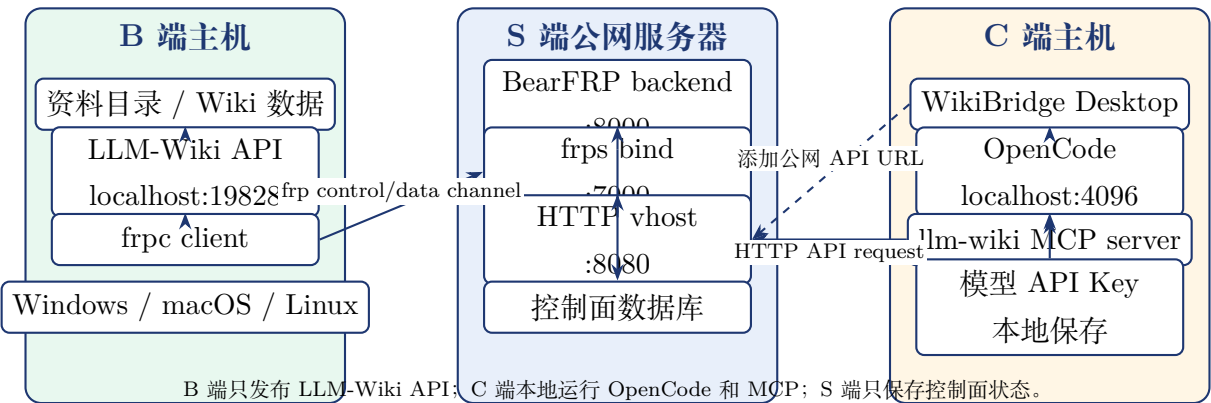


图 4: WikiBridge B/S/C 部署图

3.7.1 B 端部署

B 端部署 LLM-Wiki API 和 frpc。LLM-Wiki API 默认监听本地端口，例如 19828，frpc 只将该 API 发布到 S 端公网入口。B 端不部署对外开放的 OpenCode 页面，也不向 C 端共享模型 API Key。

3.7.2 S 端部署

S 端部署在具有公网 IP 或公网域名的 Linux 主机上，包含 BearFRP backend、frps、HTTP vhost 和控制面数据库。HTTP 子域名模式需要将通配域名解析到 S 端；若不具备域名条件，应使用 TCP 端口模式作为替代。

3.7.3 C 端部署

C 端部署 WikiBridge Desktop、OpenCode 和 llm-wiki MCP server。OpenCode 默认监听本地端口，例如 4096。C 端用户保存自己的模型配置，添加远程知识库 API URL 后，通过 MCP 工具访问 B 端 LLM-Wiki API。

3.8 异常处理与可靠性设计

异常场景	检测方式	处理策略
LLM-Wiki API 不可达	B/C 端健康检查失败或连接超时	提示检查 B 端 LLM-Wiki 服务、端口和防火墙，不创建远程知识库。
HTTP 子域名不可解析	C 端访问公网 URL 失败或域名解析异常	提示配置真实通配域名，或切换 TCP 发布模式。
Token 过期或无权限	远程 API 返回 401/403	提示重新登录、更新 token 或联系 B 端发布者。
隧道断开	frpc 心跳或 frps 状态异常	自动重连并在界面展示状态；多次失败后提示检查网络和 S 端配置。
OpenCode 未就绪	本地 4096 健康检查失败	提示启动 OpenCode、检查端口占用或查看 sidecar 日志。
MCP 注册失败	OpenCode MCP 列表缺少 llm-wiki-* 或工具调用失败	回滚注册状态，提示检查 MCP 命令、入口文件、环境变量和远程 URL。
模型 API 配置缺失	OpenCode 对话返回模型配置错误	提示在 C 端模型设置中保存 provider、model 和 API Key。
KB 安全策略命中	工具调用请求写入、删除或非白名单 MCP	拒绝执行并记录审计日志，提示当前为 KB 只读模式。

3.9 测试设计

课程设计原型阶段建议按以下场景验收：

1. B 端 API 测试：启动 LLM-Wiki API 后，确认 /api/v1/health、项目列表、文件读取和搜索接口可用。
2. BearFRP 发布测试：创建 HTTP/TCP 代理，确认 frpc 连接 frps，公网 API URL

可访问。

3. 通配域名测试：HTTP 子域名模式下验证域名解析；缺少通配域名时确认系统给出明确错误或降级建议。
4. C 端远程知识库测试：粘贴公网 API URL 后，确认 Desktop 能完成健康检查、保存远程知识库并列出项目。
5. OpenCode 启动测试：确认 C 端可启动本地 OpenCode，创建 KB Chat Session。
6. MCP 注册测试：确认 `llm-wiki-*` MCP 出现在 OpenCode 可用工具列表，并能调用 `read_file` 和 `search`。
7. 安全边界测试：确认 B 端未暴露完整 OpenCode，C 端模型 Key 不上传到 S 端，KB 只读模式和 MCP 白名单生效。
8. 异常测试：模拟远程 API 不可达、Token 错误、OpenCode 未就绪、MCP 注册失败和模型配置缺失，确认界面提示清晰且系统可恢复。

结论

WikiBridge 的设计目标是验证 “B 端 LLM-Wiki API 发布 + S 端 BearFRP/frps 转发 + C 端 OpenCode/MCP 消费” 的端到端可行性。系统通过 B/S/C 职责分离和 Agent 能力本地化，避免把 B 端完整 OpenCode 暴露到公网，也避免把 C 端模型 Key 交给 B 端或 S 端。后续实现应优先保证 B 端 API 发布、C 端远程知识库添加、OpenCode KB Session 和 `llm-wiki` MCP 工具调用闭环，再逐步完善 token 鉴权、限速策略、跨平台打包和自动化测试。

第四部分

测试说明书

(终版)

文档变更历史记录

序号	变更日期	变更人员	变更内容详情描述	变更后的版本号
1	2026 年 6 月 1 日	项目组	启动文档编写，形成初始项目背景、范围和阶段性内容。	V0.1
2	2026 年 6 月 18 日	项目组	补充核心功能、架构、测试任务和阶段性实现材料。	V0.9
3	2026 年 6 月 26 日	项目组	根据最终联调结果整理终版文档并纳入分组实习报告，补充测试方法、缺陷矩阵和个人贡献评分表。	V1.0

文档编号: WikiBridge-TAR-1.0

WikiBridge

测试分析报告

零代码本地知识库 API 分享与 OpenCode/MCP 消费平台

项目名称: WikiBridge

文档版本: V1.0

参考模板: 测试分析报告 (GB8567-88)

编写人员: 项目组

适用阶段: 课程设计测试分析与验收评审

日期: 2026 年 6 月 26 日

文档变更历史记录

序号	日期	版本	变更人员	变更内容
1	2026 年 6 月 1 日	V0.1	项目组	启动测试准备工作，明确测试对象、测试环境和课程项目验收关注点。
2	2026 年 6 月 18 日	V0.2	项目组	根据阶段性实现整理 T-01 至 T-09 测试任务、黑盒/白盒测试范围和自动化测试入口。
3	2026 年 6 月 25 日	V0.9	项目组	根据 MCP 架构调整同步测试重点：B 端只发布 LLM-Wiki API, C 端本地 OpenCode + MCP 消费远程知识库，并强化 KB 模式和 MCP 白名单测试。
4	2026 年 6 月 26 日	V1.0	项目组	根据测试文档、测试命令输出和 Playwright 截图说明整理测试分析报告，补充 Node 22 环境下 T-02、T-09 补跑结果，插入真实 UI 截图证据, 补充黑盒/白盒方法、控制流复杂度和联调缺陷记录, 形成正式测试分析结论。

目录

文档变更历史记录 I

1 引言 1

1.1 编写目的 1

1.2 背景 1

1.3 定义 1

1.4 参考资料 2

2 测试概要 2

2.1 测试环境 2

2.2 测试任务完成情况判断 3

2.3 执行命令摘要 4

2.4 测试方法与覆盖口径 5

2.5 白盒控制流图与圈复杂度摘要 6

2.6 缺陷检出矩阵 6

3 测试结果及发现 7

3.1 T-01 部署与启动测试 7

3.2 T-02 LLM Wiki API 测试 8

3.3 T-03 OpenCode KB 权限测试 8

3.4 T-04 BearFRP 用户与代理测试 10

3.5 T-05 frps 插件鉴权测试 10

3.6 T-06 自动发布 sidecar 测试 11

3.7 T-07 端到端访问测试 11

3.8 T-08 异常与安全测试 15

3.9 T-09 桌面端知识库闭环测试 16

4 对软件功能的结论 19

4.1 功能 F-01: 部署与统一入口 19

4.2 功能 F-02: LLM-Wiki 知识库服务 19

4.3 功能 F-03: OpenCode KB 权限隔离 19

4.4 功能 F-04: BearFRP 发布控制面 19

4.5	功能 F-05: 自动发布 sidecar	19
4.6	功能 F-06: 桌面端知识库闭环	19
5	分析摘要	20
5.1	能力摘要	20
5.2	缺陷和限制	20
5.3	建议	21
5.4	评价	22
6	测试资源消耗	22
A	截图证据清单	22
B	重点回归用例清单	23

1、引言

1.1 编写目的

本文档用于汇总 WikiBridge 软件系统的测试执行情况、测试结果、功能结论、遗留问题、改进建议和资源消耗。本文档参考 GB8567-88 《测试分析报告》模板编写，作为课程设计验收、项目组质量评价和后续回归测试的依据。

本文档重点回答两个问题：

1. 测试任务原始文档中规划的 T-01 至 T-09 测试任务是否已经执行并形成证据。
2. 当前版本是否满足 WikiBridge 课程设计演示和验收所需的核心质量要求。

1.2 背景

被测软件系统为 WikiBridge。系统当前架构基线为：B 端仅发布 LLM-Wiki API，S 端提供 BearFRP/frps 控制面与公网转发，C 端本地运行 OpenCode 并通过 llm-wiki MCP server 消费远程知识库。

测试对象包括：

- LLM-Wiki API 与 MCP server。
- OpenCode KB 模式及权限边界。
- BearFRP 用户、代理和 frps 插件。
- 自动发布 sidecar。
- Docker Compose 集成栈。
- WikiBridge Desktop 桌面端系统测试和 Tauri/Rust 合同测试。
- 端到端浏览器 UI 与截图证据。

1.3 定义

术语	定义
T-01 至 T-09	测试文档中定义的九类测试任务。
LLM-Wiki	本地知识库编译与 API 服务，提供健康检查、项目、文件、搜索和图谱能力。
OpenCode KB 模式	OpenCode 的知识库模式，用于限制文件访问、终端、PTY、MCP 和配置变更等能力。

BearFRP	本项目中的 frp 控制面，负责用户、代理、端口/子域名和脚本生成。
frpc/frps	frp 客户端和服务端。
MCP	Model Context Protocol, 用于将 LLM-Wiki API 注册为 Open-Code 可调用工具。
real-ui	Playwright 通过真实浏览器页面断言并截图。
version-gated	受 Node 或工具链版本限制而跳过的测试路径。

1.4 参考资料

序号	文件或资料	说明
1	测试任务原始文档	规划 T-01 至 T-09 测试任务。
2	测试命令与脚本输出整理	汇总本地测试命令和关键输出。
3	Playwright 人工测试截图说明	汇总真实 UI 截图采集结果。
4	测试脚本说明	说明自动化测试入口和执行方式。
5	可执行测试脚本	执行 T-01 至 T-09 相关自动化检查。
6	截图采集脚本	采集端到端 UI 截图证据。
7	OpenCode KB 模式说明	说明 KB 模式限制和验证点。
8	LLM-Wiki MCP server 说明	说明 MCP server 工具和配置方式。

2、测试概要

2.1 测试环境

项目	内容
测试目录	WikiBridge 项目工作区
操作系统	Linux 测试主机
Docker	Docker version 29.4.3, build 055a478

Docker Compose	Docker Compose version v5.1.3
默认 Node	v18.19.1
补跑 Node	v22.13.1, 路径 /opt/node-v22.13.1-linux-x64/bin
npm	9.2.0 / 10.9.2
Python	3.11.7
Bun	1.3.14
Rust/Cargo	已安装, Tauri/Rust 合同测试可执行
本地入口	http://127.0.0.1:18080
BearFRP 控制面	http://127.0.0.1:8000
BearFRP 发布入口	http://127.0.0.1:52600
截图目录	测试分析报告截图素材目录

2.2 测试任务完成情况判断

结论：测试任务原始文档中 T-01 至 T-09 的主体测试任务已经完成，并已形成命令输出或截图证据。当前版本满足课程设计演示和验收的核心闭环要求。

T-03 曾出现过测试脚本断言与当前实现口径不一致的情况：脚本原先预期 /vcs 和 /vcs/diff 在 KB 模式下返回 403，但当前 OpenCode 实现和源码测试要求这两个 VCS 读接口返回 HTTP 200 且内容为空对象或空数组，写入型 /vcs/apply 仍被 403 拒绝。现已将 test-tasks.mjs 的 T-03 黑盒断言同步为当前实现口径：/vcs 返回 {}、/vcs/diff 返回 []、/vcs/apply 返回 403。同步后 T-03 白盒与黑盒检查均通过。

标识	测试名称	完成状态	主要证据	说明
T-01	部署与启动测试	完成	test-tasks.mjs --full --blackbox	Compose 配置、nginx 入口、LLM-Wiki bridge health、BearFRP online API 均通过。

T-02	LLM Wiki API 完成测试	Node 22 补跑、T-02/T-07 黑盒输出	LLM-Wiki mocked tests 1548 个用例通过; MCP 14 个用例通过; API 黑盒 health/projects/files/search/graph 通过。
T-03	OpenCode KB 完成权限测试	Bun KB guard/server 测试、T-03 截图、T-03 黑盒补跑	白盒 17 个用例通过; VCS 读接口返回空结果、VCS apply 返回 403 的黑盒检查通过。
T-04	BearFRP 用户 完成与代理测试	pytest 27 passed	注册、登录、充值、代理创建、重复名称、非法输入等覆盖通过。
T-05	frps 插件鉴权测试 完成	pytest 10 passed	token、子域名、Ping、CloseProxy 等插件鉴权路径通过。
T-06	自动发布 side-car 测试 完成	测试命令输出整理	Python 语法检查通过; --full --docker 下 side-car image build 通过。
T-07	端到端访问测试 完成	黑盒输出与 real-ui 截图	本地入口、BearFRP 发布入口、/llm-wiki 页面、文件、搜索、图谱均通过。
T-08	异常与安全测试 完成	pytest 37 passed、XSS 截图	未认证、未知路径、超大请求体、XSS dialog 监听等通过。
T-09	桌面端知识库闭环测试 完成	Node 22 Playwright、Rust 合同测试、T-09 截图	Desktop Playwright 16 个用例通过; Rust 31 passed、1 ignored; 四张桌面端截图通过。

2.3 执行命令摘要

测试范围	命令摘要	结果
语法检查	node --check scripts/test-tasks.mjs	退出码 0。
空白检查	git diff --check 相关测试脚本	退出码 0。

T-02 补跑	PATH="/opt/node-v22.13.1-linux-x64/bin:\$PATH" node scripts/test-tasks.mjs --full --task T-02	24 passed、0 skipped、 0 failed。
T-03 白盒和黑盒补跑	node scripts/test-tasks.mjs --full --blackbox --task T-03 --base-url http://127.0.0.1:18080	5 passed、0 skipped、 0 failed。
T-06 Docker 构建	node scripts/test-tasks.mjs --full --docker --task T-06	2 passed、0 skipped、 0 failed。
T-09 补跑	PATH="/opt/node-v22.13.1-linux-x64/bin:\$PATH" node scripts/test-tasks.mjs --full --task T-09	24 passed、0 skipped、 0 failed。
截图采集	PATH="/opt/node-v22.13.1-linux-x64/bin:\$PATH" node scripts/capture-manual-ui-screenshots.mjs	T-03、T-07、T-08、T-09 共 12 张 real-ui 截图通过。

2.4 测试方法与覆盖口径

本轮测试采用黑盒、白盒和端到端联调结合的方式。黑盒测试主要面向用户可观察接口和页面，白盒测试主要面向权限边界、代理状态流转和 MCP 工具调用逻辑。下表列出本报告中与个人总结可追溯引用关系最强的测试方法。

方法类型	具体方法	使用位置	检出或验证结果
黑盒	等价类划分	T-04 BearFRP 用户与代理测试；T-02 LLM-Wiki API 测试	区分合法/非法名称、Token、URL、路径和代理类型。
黑盒	边界值分析	T-04 余额、流量额度、TCP 端口范围；T-09 Node 版本门控	检出并复盘 L-07 余额刚好用尽语义问题，暴露 L-02 Node 版本限制。
黑盒	状态迁移测试	T-04/T-05 代理 active、offline、deleted、stopped_by_admin 状态	发现 L-08 删除后统计与列表口径不一致问题。

黑盒	端到端场景测试	T-07、T-09	验证 B 端发布 API、S 端转发、C 端 Open-Code/MCP 消费远程知识库闭环。
白盒	分支覆盖	T-03 KB guard/server tests; T-04 BearFRP pytest	覆盖路径穿越、VCS 写入拒绝、余额不足、子域名冲突等分支。
白盒	控制流复杂度检查	CFG-01 至 CFG-04	识别 create/update 代理函数职责偏多，作为后续重构依据。

2.5 白盒控制流图与圈复杂度摘要

根据源码分支结构手工抽取 4 个控制流图。完整图形在代码审查和个人总结中按函数名追溯，本文档给出测试设计所需的节点、判定和圈复杂度摘要。

编号	被测函数或流程	主要判定节点	圈复杂度	对应测试
CFG-01	create_proxy	名称非空、余额、重名、连接数、TCP/HTTP/XTCP 类型、端口/子域名分支	8	T-04
CFG-02	update_proxy	代理归属、名称、local_ip/local_port、流量额度、HTTP 子域名、XTCP visitor、高级配置	9	T-04
CFG-03	poller._update_proxy_status	非 active 跳过、已用流量是否达到额度	3	T-05、L-07
CFG-04	llm_wiki MCP 工具调用处理	工具名分发、MCP 启用检查、project_id 参数、读取/搜索/图谱/重扫分支	8	T-02、T-07

2.6 缺陷检出矩阵

编号	缺陷或限制摘要	黑盒	白盒	联调/CI	当前状态
L-01	T-03 VCS 读接口测试脚本与实现口径不一致	是	是	否	已更新脚本
L-02	默认 Node 18.19.1 低于 Desktop Playwright/Vite 要求	是	否	是	已用 Node 22 补跑
L-03	npm optional native binding 缺失	是	否	是	已补依赖
L-04	OpenCode 容器 healthcheck 与真实入口状态不一致	是	否	是	待修正
L-05	公开部署前默认密码和 token 必须替换	是	是	否	部署前必须处理
L-06	real-LLM、HTTP 子域名和公网访问依赖外部环境	是	否	是	按环境配置
L-07	BearFRP 余额刚好用尽时代理运行语义不清	是	是	是	已按代理额度口径修正
L-08	删除连接后统计与列表展示口径不一致	是	否	是	待明确 active/history 口径

3、测试结果及发现

3.1 T-01 部署与启动测试

检查项	输入或命令	实际结果	判定
Docker Compose 配置	<code>docker compose config --quiet</code>	配置解析通过。	通过
nginx OpenCode entry	<code>GET /global/health</code>	HTTP 200，健康响应正常。	通过
LLM-Wiki bridge health	<code>GET /instance/llm-wiki/health</code>	HTTP 200。	通过

BearFRP online GET /api/show/online HTTP 200, 返回对象。[通过](#)

API

发现：系统能够按 Compose 方式启动并形成 nginx 单入口。当前 docker compose ps 中 OpenCode 容器曾显示 unhealthy，但通过 nginx 的 /global/health 和真实页面访问均通过，说明容器级 healthcheck 与实际入口健康状态存在口径差异，建议后续修正 healthcheck 规则。

3.2 T-02 LLM Wiki API 测试

检查项	输入或命令	实际结果	判定
mocked tests	npm run test:mocks	Node 22 环境下 105 个测试文件、1548 个测试用例通过。	通过
MCP tests	npm run mcp:test	14 个 MCP 测试全部通过。	通过
health	GET /api/v1/health	HTTP 200。	通过
projects	GET /api/v1/projects	HTTP 200, 返回项目数组。	通过
project files	GET /projects/:id/files	HTTP 200, 返回文件数据。	通过
search	POST /projects/:id/search	HTTP 200, 返回搜索结果。	通过
graph	GET /projects/:id/graph	HTTP 200, 返回图谱数据。	通过

过程记录：最初在默认 Node 18.19.1 环境中，llm_wiki mocked tests 被脚本按 Node 版本门控跳过；切换到 Node 22.13.1 后首次执行因 npm optional dependency 缺少 @rolldown/binding-linux-x64-gnu 失败，执行 npm install --include=optional 后补跑通过。该问题属于本地依赖安装不完整，不是业务断言失败。

发现：LLM-Wiki API 和 MCP server 的主要接口、错误处理、Token 传递、搜索和图谱解析能力满足测试文档要求。

3.3 T-03 OpenCode KB 权限测试

检查项	输入或命令	实际结果	判定
KB guard tests	<code>bun test</code> <code>test/kb/guard.test.ts</code> <code>test/kb/server.test.ts</code>	17 pass, 0 fail。	通过
私有目录读写	单元测试覆盖	私有目录可读写。	通过
公开 Wiki 只读	单元测试覆盖	公开 Wiki 可读，不可写。	通过
路径穿越	单元测试覆盖	<code>../</code> 和项目外路径被拒绝。	通过
软链接逃逸	单元测试覆盖	<code>realpath</code> 后拒绝。	通过
Shell/PTY/MCP/Config tests 覆盖		禁用或返回拒绝。	通过
KB meta/UI 截图 1 图		页面暴露 KB mode meta 并显示 KB UI。	通过
VCS 读接口黑盒断言	<code>/vcs</code> 、 <code>/vcs/diff</code>	HTTP 200，返回空对象或空数组。	通过
VCS 写接口黑盒断言	<code>/vcs/apply</code>	KB 模式下返回 HTTP 403。	通过

发现：KB 模式的核心安全边界已经通过白盒测试、黑盒测试和真实 UI 证据。VCS 读接口的当前实现是“保留读接口但返回空结果”，测试脚本已同步为该口径，并额外检查 `/vcs/apply` 必须返回 403，以覆盖写入型 VCS 能力禁用要求。

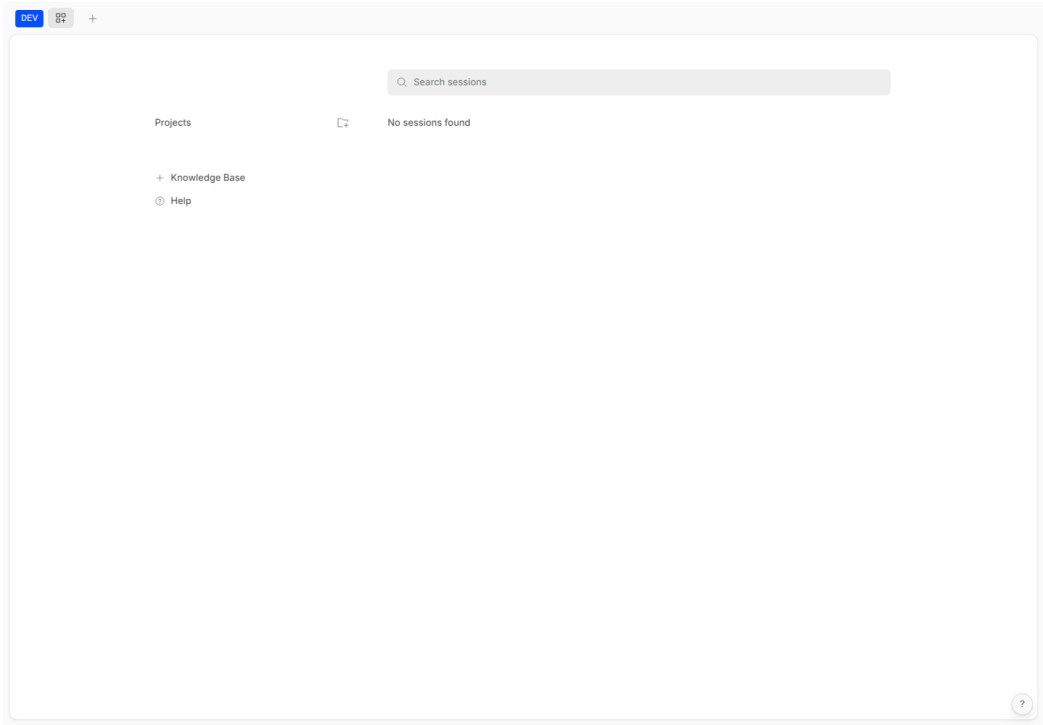


图 1: T-03 OpenCode KB 模式页面与 meta 标识

3.4 T-04 BearFRP 用户与代理测试

检查项	输入或命令	实际结果	判定
BearFRP API coverage	python -m pytest -q tests/test_api.py	27 passed。	通过

本项采用等价类、边界值和状态迁移测试。等价类包括合法/非法代理名称、TCP/HTTP/XTCP 类型、合法/非法子域名和认证状态；边界值包括余额刚好等于流量额度、端口池上下界、TCP 多端口数量上限；状态迁移覆盖 active、deleted、stopped_by_admin 和脚本重新获取。

发现：用户注册、登录、充值、代理创建、脚本生成、重复名称、非法输入和删除等控制面核心路径已通过当前 pytest 覆盖。联调阶段曾暴露两个与用户流程直接相关的问题：L-07 余额刚好用尽后代理被轮询停用，根因是账户余额和代理已分配额度语义混用；L-08 删除代理后用户统计与代理列表默认展示口径不一致，容易让前端或用户误判删除失败。当前整合仓库中轮询停用逻辑已按 traffic_limit_mb 和 traffic_used_bytes 判断，不再直接用 balance_mb <= 0 停用已创建代理；删除列表口径仍建议后续拆分“当前可管理连接”和“历史记录”。

3.5 T-05 frps 插件鉴权测试

检查项	输入或命令	实际结果	判定
plugin and poller tests	<code>python -m pytest -q tests/test_plugin_and_poller.py</code>	10 passed。	通过

发现：frps 插件能够正确处理 token、代理名、子域名、Ping 和 CloseProxy 等关键鉴权与状态同步场景。

3.6 T-06 自动发布 sidecar 测试

检查项	输入或命令	实际结果	判定
sidecar syntax	<code>Python node scripts/test-tasks.mjs --task T-06</code>	Python AST 解析通过。	通过
sidecar build	<code>image node scripts/test-tasks.mjs --full --docker --task T-06</code>	sidecar 镜像构建通过。	通过

发现：sidecar 能支持课程演示中的自动发布路径。默认轻量模式会跳过 Docker build，这是脚本配置行为；`--full --docker` 模式已实际构建成功。

3.7 T-07 端到端访问测试

检查项	输入或命令	实际结果	判定
OpenCode advertises mode	<code>page GET / KB</code>	HTTP 200，包含 <code>opencode-kb-mode</code> 。	通过
bridge project list	<code>GET /instance/llm-wiki/projects</code>	HTTP 200，返回项目数组。	通过
bridge project files	<code>GET /instance/llm-wiki/projects/:id/files</code>	HTTP 200，返回文件数组。	通过
本地入口截图	图 2	real-ui 通过。	通过
知识库页面	图 3	real-ui 通过。	通过
图谱视图	图 4	real-ui 通过。	通过
文件内容	图 5	real-ui 通过。	通过

搜索结果	图 6	real-ui 通过。	通过
BearFRP 发布入口	图 7	real-ui 通过。	通过

发现：本地入口、LLM-Wiki 页面、搜索、图谱和 BearFRP 发布入口均能形成可演示的端到端访问闭环。

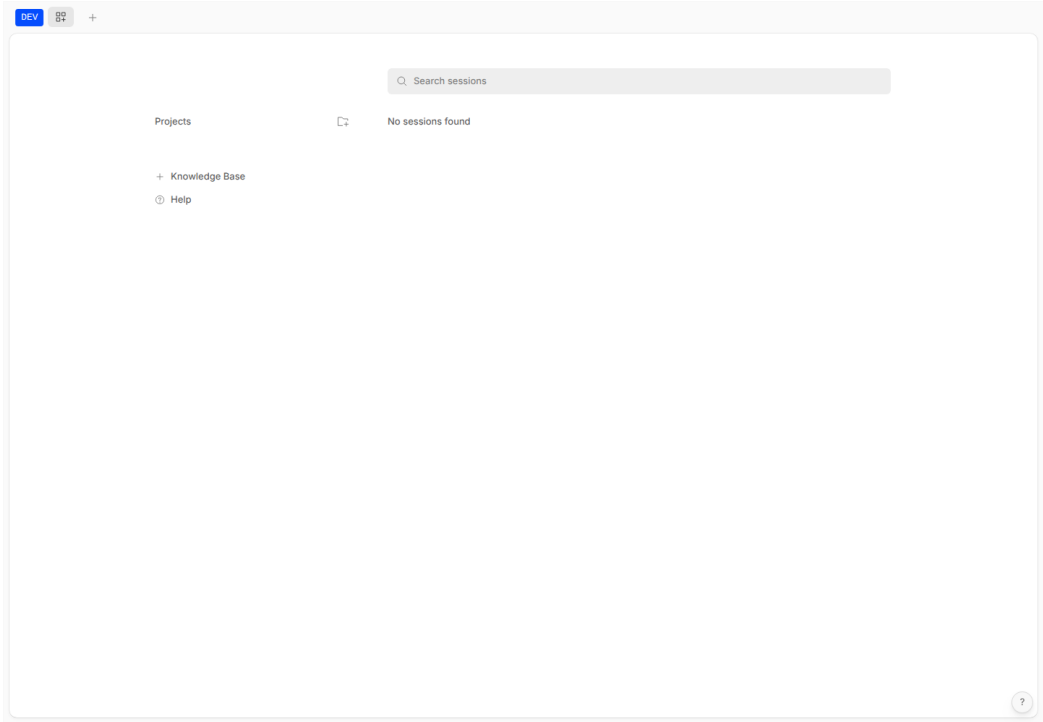


图 2: T-07 本地入口 OpenCode KB 页面

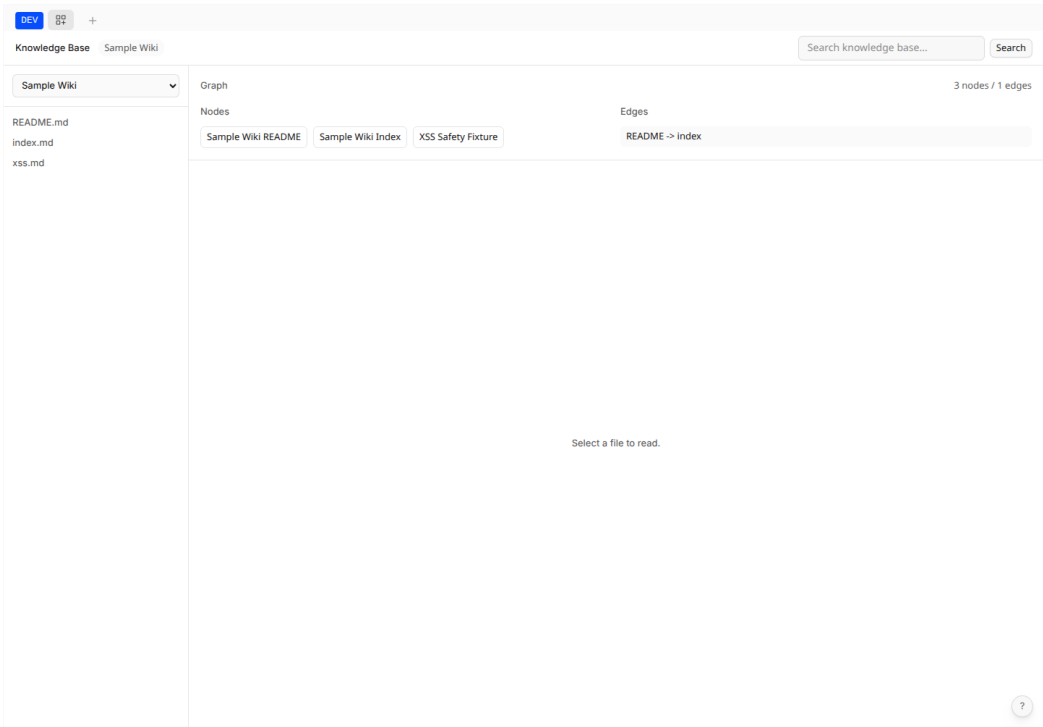


图 3: T-07 LLM-Wiki 知识库页面

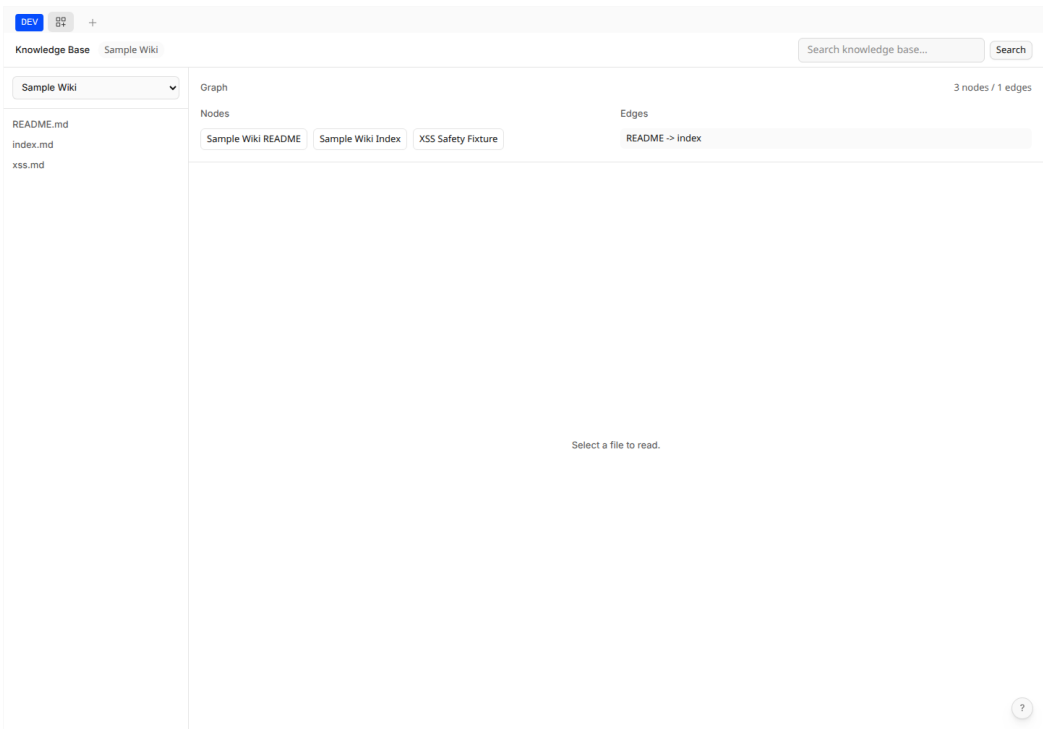


图 4: T-07 LLM-Wiki 图谱视图

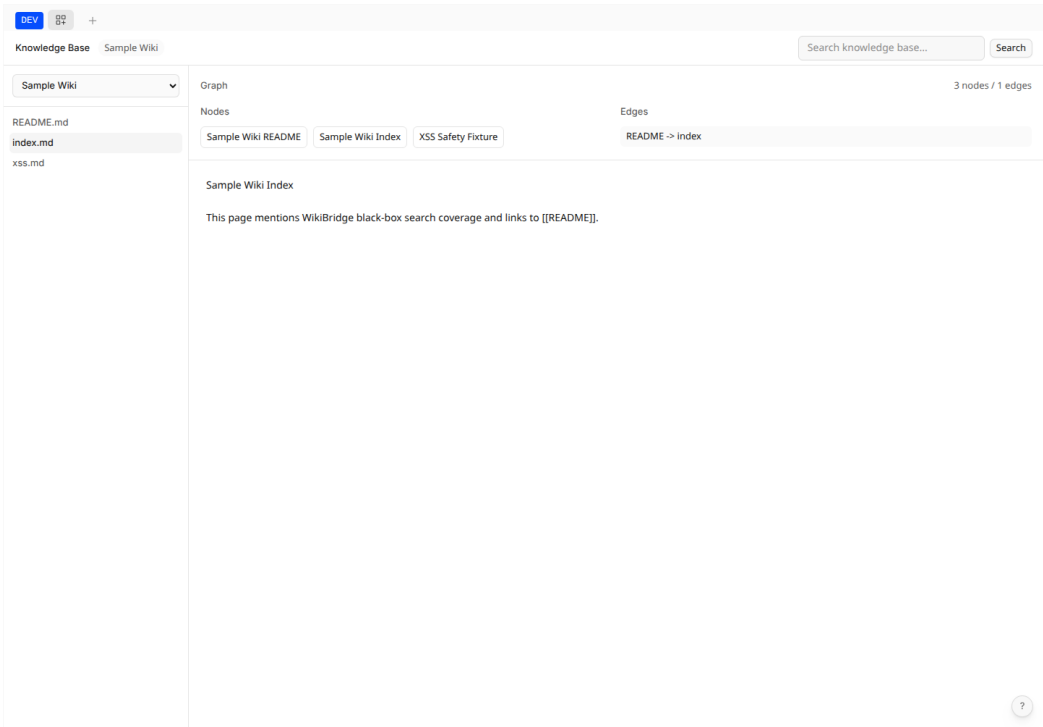


图 5: T-07 LLM-Wiki 文件内容页面

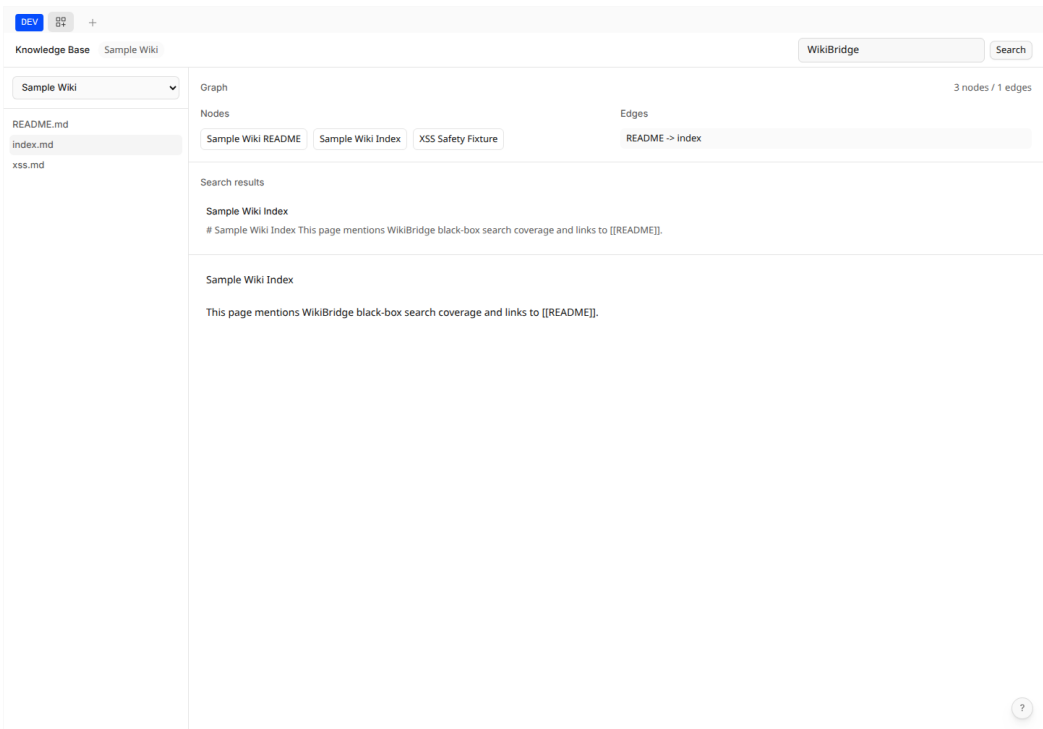


图 6: T-07 LLM-Wiki 搜索结果页面

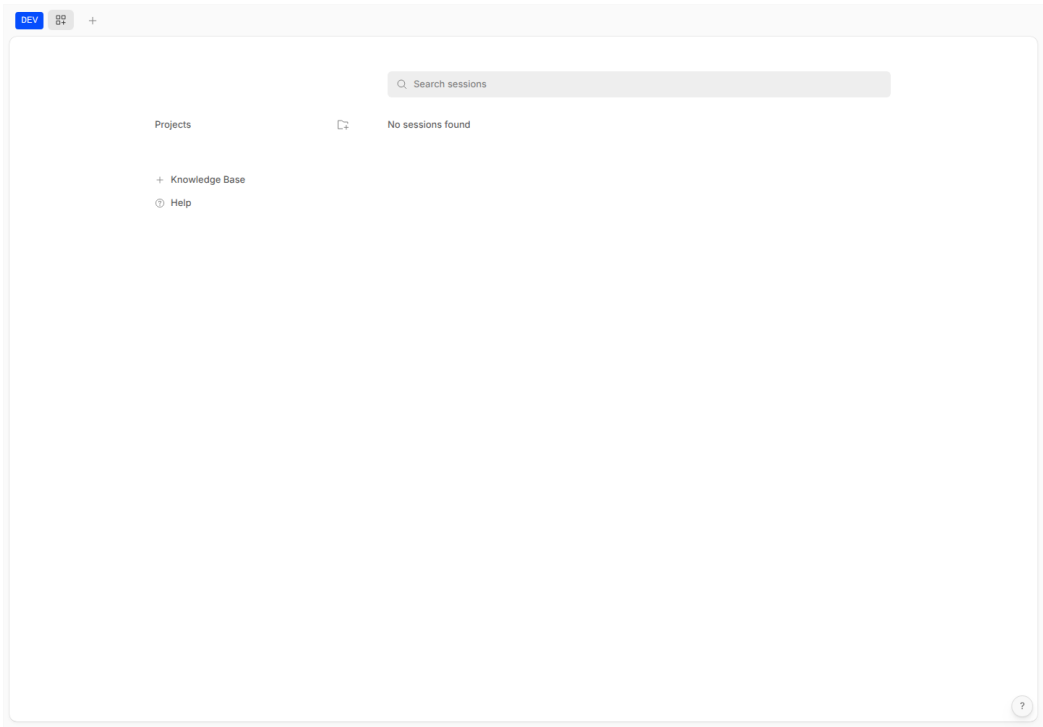


图 7: T-07 BearFRP 发布入口页面

3.8 T-08 异常与安全测试

检查项	输入或命令	实际结果	判定
BearFRP security/error cover- age	pytest tests/test_api.py tests/test_plugin_and_poller.py	37 passed。	通过
unknown BearFRP path	GET /__not_found__	HTTP 404。	通过
user endpoint re- quires auth	GET /api/user/me	HTTP 401。	通过
LLM Wiki over- sized body	超大 search body	HTTP 503，符合脚本允许状态。	通过
XSS dialog 监听	图 8	未捕获浏览器弹窗。	通过

发现：错误 token、未认证访问、未知路径、超大请求体和基础 XSS 样例均按安全边界处理。公开部署前仍需替换默认密码、frps token、LLM_WIKI_TOKEN，并配置 HTTPS。

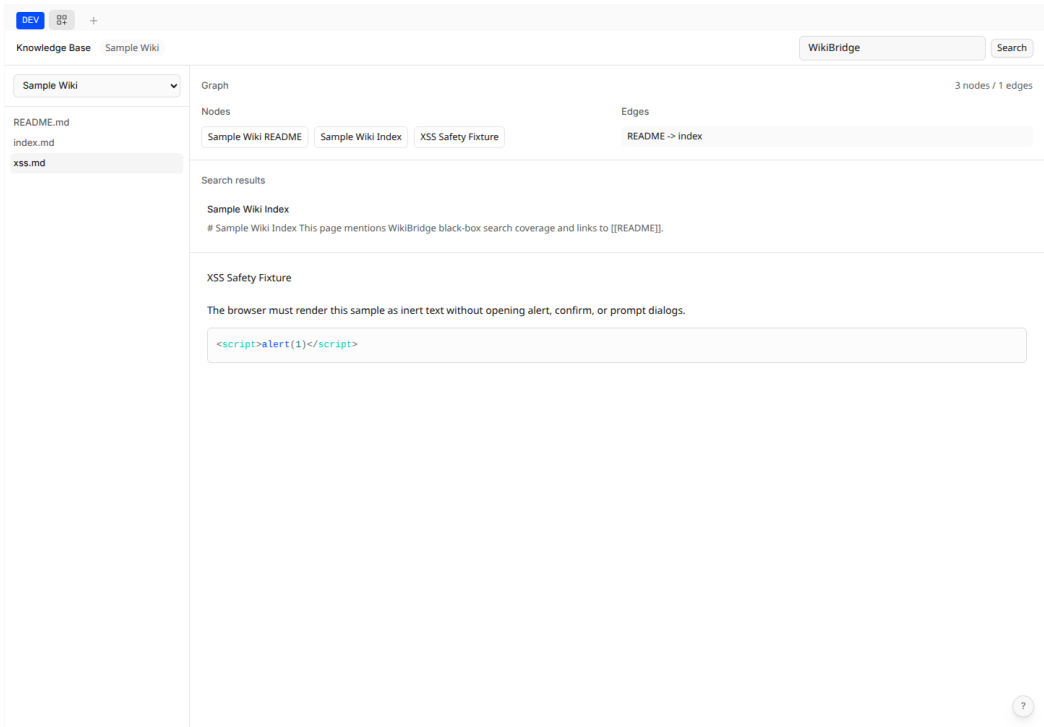


图 8: T-08 XSS 样例页面未触发浏览器弹窗

3.9 T-09 桌面端知识库闭环测试

检查项	输入或命令	实际结果	判定
Desktop Playwright tests	<code>npm run test:system</code>	Node 22 环境下 16 passed。	通过
Desktop Rust contract tests	<code>npm run test:contracts</code>	31 passed、1 ignored。	通过
项目仪表盘截图	图 9	real-ui 通过。	通过
编译准备状态截图	图 10	real-ui 通过。	通过
访问连接状态截图	图 11	real-ui 通过。	通过
消费端远程知识库截图	图 12	real-ui 通过。	通过

过程记录：T-09 最初在 Node 18.19.1 环境中被 Playwright/Vite 版本门控跳过；切换至 Node 22.13.1 后，Desktop Playwright 系统测试和 Rust 合同测试均通过，截图脚

本也补齐四张桌面端 real-ui 证据。

发现：桌面端已覆盖本地知识项目、远程知识库添加、发布连接和消费端入口等课程演示所需路径。

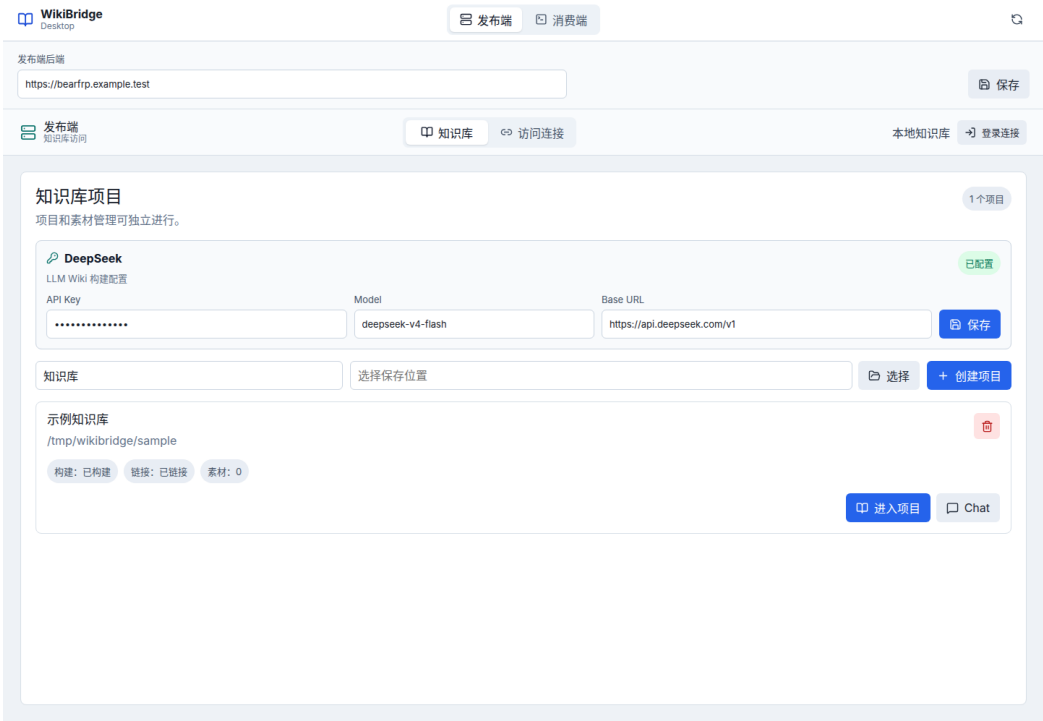


图 9：T-09 桌面端知识库项目仪表盘

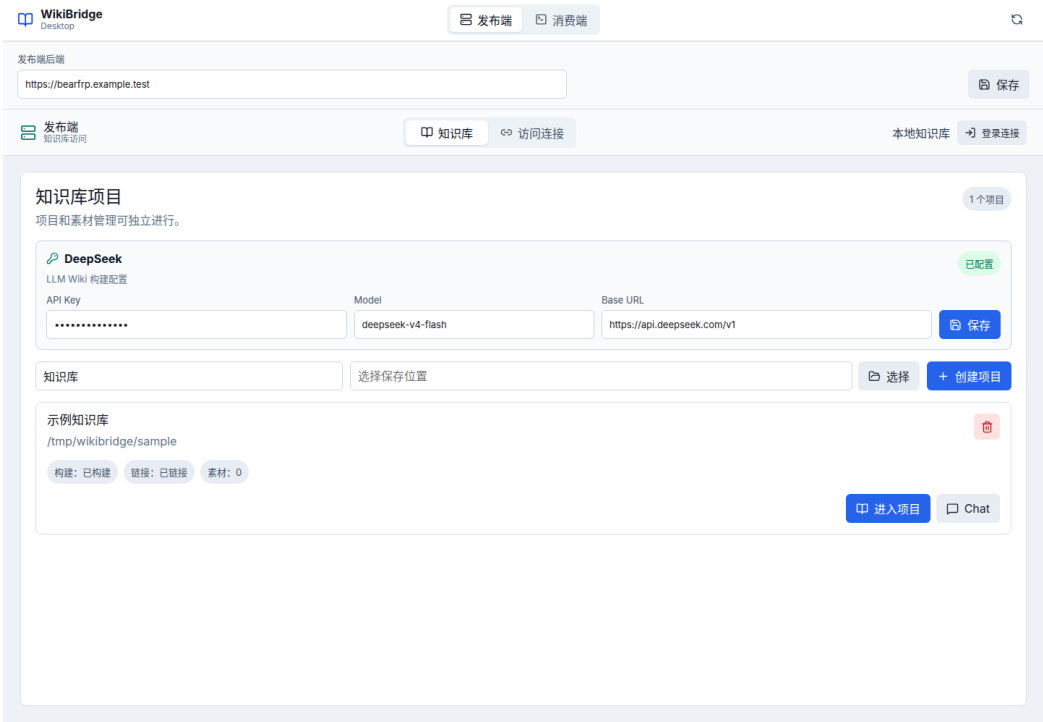


图 10：T-09 桌面端编译准备状态

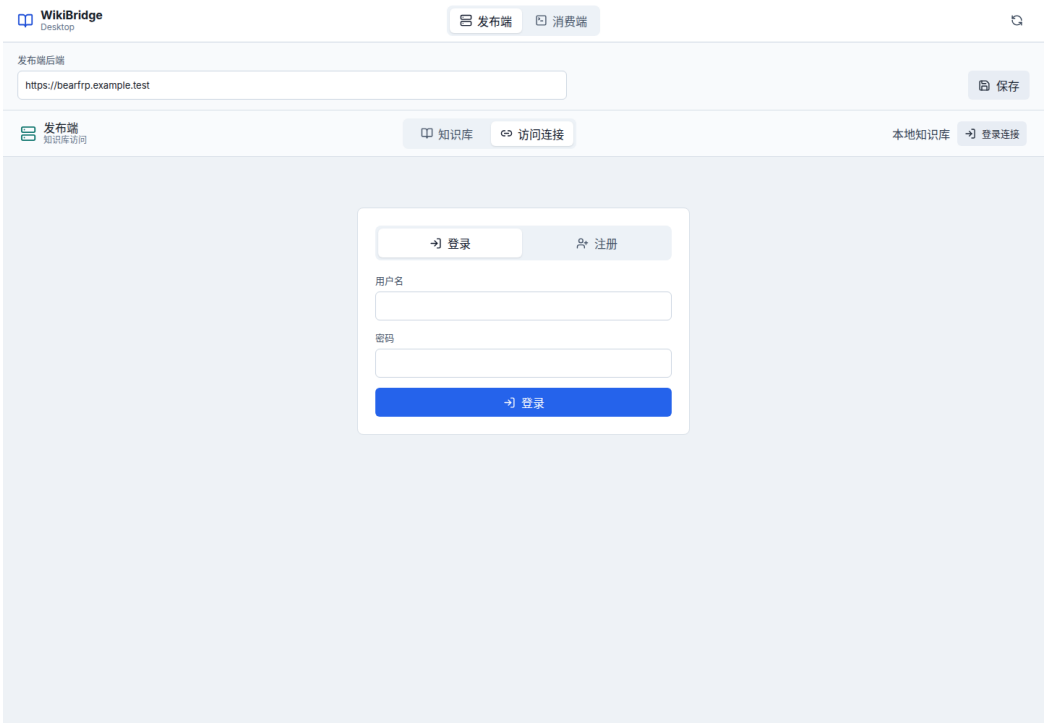


图 11: T-09 桌面端访问连接状态

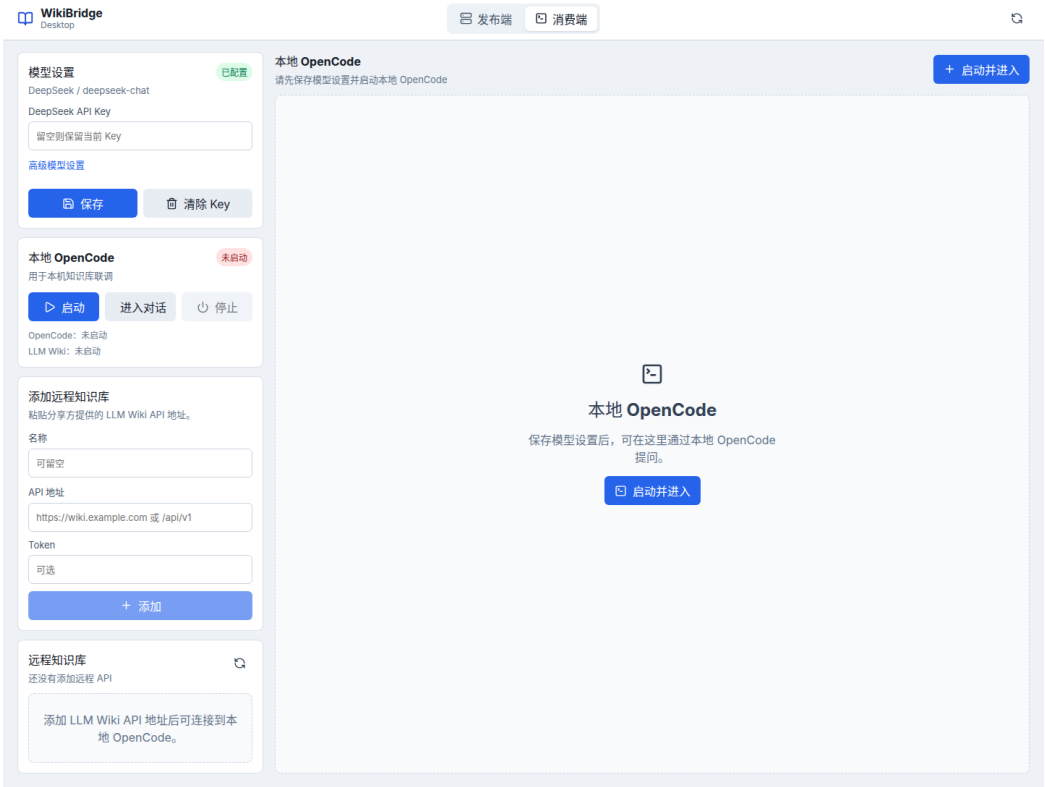


图 12: T-09 桌面端消费端远程知识库入口

4、对软件功能的结论

4.1 功能 F-01：部署与统一入口

结论：通过。系统能够通过 Docker Compose 形成可访问的 nginx 单入口，并转发 OpenCode KB 页面和 LLM-Wiki bridge API。

限制：OpenCode 容器级 healthcheck 与真实入口健康状态存在口径差异，建议后续修正容器健康检查。

4.2 功能 F-02：LLM-Wiki 知识库服务

结论：通过。LLM-Wiki 的健康检查、项目、文件、搜索、图谱、mocked tests 和 MCP server 测试均通过。

限制：real-LLM 能力依赖外部模型服务和 API Key，本轮主要验证 mock、API、MCP 和黑盒项目级能力。

4.3 功能 F-03：OpenCode KB 权限隔离

结论：通过。KB Guard、server 权限、路径隔离、Shell/PTY/MCP/Config 禁用、VCS 读接口脱敏、VCS 写接口拒绝和 KB meta/UI 证据均通过。

限制：VCS 读接口采用“返回空结果”而非“直接 403”的安全口径，后续需求和测试文档需保持一致。

4.4 功能 F-04：BearFRP 发布控制面

结论：通过。BearFRP 用户、代理、frps 插件和安全错误路径 pytest 均通过，BearFRP 发布入口截图通过。

限制：对外部署仍依赖公网端口、防火墙、HTTP vhost、DNS 或 TCP 端口配置。

4.5 功能 F-05：自动发布 sidecar

结论：通过。sidecar 语法检查和 Docker 镜像构建通过，能够支撑课程演示的一键发布链路。

限制：首次运行可能依赖 frpc 下载或镜像缓存，离线环境需提前准备二进制或缓存。

4.6 功能 F-06：桌面端知识库闭环

结论：通过。Desktop Playwright 系统测试、Rust 合同测试和四张 real-ui 截图均通过。

限制：测试截图覆盖的是系统测试模式下的桌面前端入口；真实 Tauri 打包后的跨

平台表现仍需后续按提交平台补测。

5、分析摘要

5.1 能力摘要

本轮测试证明 WikiBridge 当前版本具备以下能力：

- 1. 能以 Docker Compose 方式启动主要服务，并通过 nginx 单入口访问。
- 2. 能通过 LLM-Wiki API 提供项目、文件、搜索和图谱能力。
- 3. 能通过 llm-wiki MCP server 将远程知识库能力接入 OpenCode。
- 4. 能通过 OpenCode KB 模式限制危险文件访问、终端、PTY、MCP 和配置变更。
- 5. 能通过 BearFRP 控制面和 frps 插件创建、鉴权和管理发布代理。
- 6. 能通过 sidecar 自动发布演示入口。
- 7. 能通过浏览器真实 UI 展示知识库页面、图谱、文件内容、搜索结果和 XSS 不执行证据。
- 8. 能通过 Desktop 完成本地知识项目、发布连接和远程知识库消费端入口展示。

5.2 缺陷和限制

编号	缺陷或限制	严重性	影响	状态
L-01	T-03 汇总黑盒脚本曾要求 VCS 读接口返回 403，但当前实现返回 200 空结果。	低	曾造成全量汇总脚本 2 个失败断言。	已更新脚本
L-02	默认 Node 18.19.1 低于 Desktop Playwright/Vite 要求。	中	不切换 Node 时 T-09 系统测试和截图会被跳过。	已用 Node 22 补跑
L-03	llm_wiki/node_modules 曾缺少 npm optional native binding。	中	Node 22 下首次执行 mocked tests 失败。	已修复补跑
L-04	OpenCode 容器级 healthcheck 曾显示 unhealthy，但入口健康检查通过。	低	可能影响部署状态判断。	待修正

L-05	对外部署前默认密码、frps token、LLM_WIKI_TOKEN 和 HTTPS 仍需按生产环境设置。	高	公开暴露时存在安全风险。	部署前必须处理
L-06	real-LLM、HTTP 子域名和公网访问依赖外部模型服务、DNS、防火墙和端口配置。	中	可能影响非本地演示环境稳定性。	按环境配置
L-07	BearFRP 余额刚好用尽时，旧口径会把账户余额为 0 解释为已创建代理不可运行。	中	可能导致用户刚创建成功的代理被轮询置为 stopped_by_admin。	已按代理流量额度口径修正
L-08	普通用户删除连接后，用户统计与代理列表默认展示 deleted 记录的口径不一致。	低	前端和用户可能误以为连接未删除，自动化校验也容易误判。	待明确 active/history 口径

5.3 建议

编号	建议	紧迫程度	预计工作量
S-01	保持 test-tasks.mjs 中 T-03 VCS 黑盒断言与当前实现一致：/vcs 预期 \{\}\、/vcs/diff 预期 []、/vcs/apply 预期 403。	中	0.5 人日
S-02	在提交或答辩环境统一使用 Node 22.13.1 或 Node 20.19+，避免 Desktop 测试被版本门控跳过。	高	0.5 人日
S-03	将 npm install --include=optional 或等价依赖安装步骤写入测试准备说明。	中	0.5 人日
S-04	修正 OpenCode 容器 healthcheck，使 Compose 状态与 /global/health 一致。	中	1 人日
S-05	公开部署前强制修改 OpenCode 密码、BearFRP 管理员密码、frps token 和 LLM_WIKI_TOKEN，并配置 HTTPS。	高	0.5 人日
S-06	保留 T-03/T-07/T-08/T-09 real-ui 截图作为最终验收证据，并在答辩前重新生成一轮。	中	0.5 人日

S-07	为 L-07/L-08 增补回归断言：余额扣减后代理是否 中 按已分配流量运行，普通用户列表是否默认只展示 当前可管理连接。	0.5 人日
------	--	--------

5.4 评价

从测试计划覆盖范围和当前执行结果看，WikiBridge 已完成测试文档中 T-01 至 T-09 的主体测试任务。系统可以用于课程设计演示和验收。

当前没有发现影响主链路的阻塞级缺陷。主链路包括：B 端构建并通过 LLM-Wiki API 暴露知识库，S 端通过 BearFRP/frps 发布入口，C 端本地 OpenCode + MCP 消费远程知识库。需要在最终提交前处理或说明的事项主要是 Node 版本要求、公开部署安全配置和容器 healthcheck 口径。

6、测试资源消耗

资源类型	数量或时间	说明
测试人员	1-2 人	执行脚本、补跑环境、整理截图和报告。
开发协助	1 人	解释当前 OpenCode VCS 读接口和 KB 模式实现口径。
测试主机	1 台	Linux + Docker Compose 环境。
浏览器	Chromium / Playwright	用于 real-ui 截图和 Desktop 系统测试。
命令行测试时间	约 1 小时	包括 T-02、T-03、T-09 补跑和全量汇总。
截图采集时间	约 10 分钟	生成 T-03、T-07、T-08、T-09 共 12 张截图。
报告整理时间	约 1.5 小时	对照测试文档、命令输出和截图说明编写。

A、截图证据清单

测试项	截图文件	结果	证据类型
T-03	OpenCode KB 模式页面与 meta 标识截图	PASS	real-ui
T-07	本地入口 OpenCode KB 页面截图	PASS	real-ui
T-07	LLM-Wiki 知识库页面截图	PASS	real-ui

T-07	LLM-Wiki 图谱视图截图	PASS	real-ui
T-07	LLM-Wiki 文件内容页面截图	PASS	real-ui
T-07	LLM-Wiki 搜索结果页面截图	PASS	real-ui
T-07	BearFRP 发布入口页面截图	PASS	real-ui
T-08	XSS 样例页面截图	PASS	real-ui
T-09	桌面端知识库项目仪表盘截图	PASS	real-ui
T-09	桌面端编译准备状态截图	PASS	real-ui
T-09	桌面端访问连接状态截图	PASS	real-ui
T-09	桌面端消费端远程知识库入口截图	PASS	real-ui

B、重点回归用例清单

编号	回归用例	预期结果
REG-01	Compose 配置解析和 nginx 健康检查	配置有效，入口 HTTP 200。
REG-02	LLM-Wiki	均返回 HTTP 200 和预期 JSON 结构。
REG-03	LLM-Wiki mocked tests	Node 20.19+ 或 Node 22 下全部通过。
REG-04	LLM-Wiki MCP tests	14 个 MCP 测试全部通过。
REG-05	KB 模式路径穿越、软链接、其它用户目录	均被拒绝。
REG-06	KB 模式 Shell/PTY/MCP/Config	禁用或返回拒绝。
REG-07	KB 模式 VCS 读接口	/vcs 返回 200 \{\}, /vcs/diff 返回 200 []。
REG-08	KB 模式 VCS apply	返回 403。
REG-09	BearFRP 用户和代理 pytest	27 passed。
REG-10	frps 插件 pytest	10 passed。
REG-11	sidecar 语法和镜像构建	均通过。
REG-12	本地入口和 BearFRP 发布入口	均可访问并展示 KB shell。
REG-13	/llm-wiki 文件、搜索、图谱 UI	real-ui 断言通过并生成截图。

REG-14	XSS fixture	页面显示脚本文本但不触发浏览器弹窗。
REG-15	Desktop Playwright system tests	16 passed。
REG-16	Desktop Rust contract tests	31 passed、1 ignored。
REG-17	BearFRP 余额边界回归	余额等于流量额度时，创建成功后代理不因账户余额为 0 被立即停用。
REG-18	BearFRP 删除列表口径回归	删除后用户统计和普通用户默认列表口径一致；如保留历史记录，需通过显式参数或管理员接口查询。

第五部分

个人总结报告

(终版)

此部分待插入小组成员个人总结报告。

第六部分

小组个人贡献评分表 (终版)

小组个人贡献评分表 (全程)

组号:B260

项目:WikiBridge 零代码本地知识库 API 分享与 OpenCode/MCP

消费平台

序号	班序号、学号	姓名	角色	贡献比	主要贡献
1	SES27、 2024302121024	张沐雷	组织工作、 BearFRP 前端、跨端联调、持续集成 CI、桌面端测试/自动化相关	待确认	负责项目组织协调、BearFRP 前端、跨端联调、持续集成 CI、桌面端测试和自动化测试流程；推动桌面端整合 BearFRP、LLM Wiki、OpenCode，补充系统测试、集成测试和格式化检查，以此修复大量 bug，降低多技术栈融合后的接口、UI 和跨平台回归风险
2		陈明德	OpenCode + MCP 接入、跨端联调、BearFRP 开发	待确认	完成 llm-wiki MCP server、OpenCode KB 接入与远程知识库联调，参与 B/S/C 架构调整，实现 BearFRP 持久化和普通用户连接配置修改，发现并复盘关键联调缺陷。
3		郭一言	Wiki 方向与 frp 联调	待确认	负责 Wiki 相关集成、桌面端发布侧材料、frp 与 LLM-Wiki 整合，参与发布链路联调和需求/计划材料整理。

4		骆贝尔	LLM-Wiki 方向	待确认	负责 LLM-Wiki 知识库生成、项目文件与搜索能力相关实现和联调支撑，提供 B 端知识库服务基础能力。
5		曾浩正	OpenCode / Chatbox 方 向	待确认	负责 OpenCode / Chatbox 知识消费方向开发，参与 KB 会话、对话入口和知识消费体验联调。
6		童济舟	OpenCode / Chatbox 方 向	待确认	参与 OpenCode / Chatbox 方向开发、知识消费链路联调和相关材料整理。

说明：贡献比待组内确认后填写；当前表格仅保留成员角色和主要方向，不作为最终贡献比例依据。班序号和学号由提交前按课程名单补齐。